

Pengembangan Arsitektur Platform DataOps dan MLOps Terintegrasi pada Kubernetes dengan Pemanfaatan *Open Source Tools*

Laporan Tugas Akhir

Oleh

Bryan P. Hutagalung
18222130



**PROGRAM STUDI SISTEM DAN TEKNOLOGI INFORMASI
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG**

Mei 2026

LEMBAR PENGESAHAN

Pengembangan Arsitektur Platform DataOps dan MLOps Terintegrasi pada Kubernetes dengan Pemanfaatan *Open Source* *Tools*

Laporan Tugas Akhir

Oleh

Bryan P. Hutagalung
18222130

Program Studi Sistem dan Teknologi Informasi
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung

Laporan Tugas Akhir ini telah disetujui dan disahkan
di Bandung, pada tanggal 25 Mei 2026

Pembimbing

Achmad Imam Kistijantoro, S.T, M.Sc., Ph.D.

NIP. 197308092006041001

PERNYATAAN ORISINALITAS

Saya yang bertanda tangan di bawah ini menyatakan dengan sesungguhnya bahwa:

1. Tugas Akhir ini adalah hasil karya saya sendiri yang dibuat dengan sebenarnya sesuai dengan kaidah ilmiah dan etika akademik.
2. Semua sumber rujukan dan data yang saya gunakan dalam penyusunan laporan tugas akhir ini, baik yang berupa kutipan langsung maupun tidak langsung, telah saya cantumkan sumbernya dengan baik dan benar sesuai dengan ketentuan penulisan laporan tugas akhir.
3. Isi laporan ini belum pernah diajukan pada program pendidikan di institusi mana pun.

Apabila di kemudian hari terbukti bahwa laporan ini melanggar hal-hal di atas, saya bersedia menerima sanksi sesuai dengan peraturan yang berlaku di Institut Teknologi Bandung.

Bandung, 25 Mei 2026

Bryan P. Hutagalung

NIM: 18222130

PERNYATAAN PENGGUNAAN AI

Saya yang bertanda tangan di bawah ini menyatakan bahwa dalam penulisan dokumen Tugas Akhir ini, saya menggunakan alat bantu kecerdasan artifisial (AI) untuk beberapa aspek tertentu di dalam proses penulisan, seperti yang tercantum dalam tabel berikut.

No.	Jenis	Alat Bantu	Tingkat	Tempat
1	Memeriksa ejaan dan tata bahasa	Grammarly	Tinggi	Semua bab
2	Pembuatan teks	Tidak ada	Tidak ada	Tidak ada
3	Pembuatan grafik, gambar, atau ilustrasi	Tidak ada	Tidak ada	Tidak ada
4	Pencarian informasi atau referensi	Tidak ada	Tidak ada	Tidak ada
5	Penyusunan bahan diskusi	Tidak ada	Tidak ada	Tidak ada
6	Penyusunan kode program	GitHub Copilot	Sedang	Lampiran A (manifest platform)
7	Penyusunan formula atau perhitungan matematis	Tidak ada	Tidak ada	Tidak ada
8	Penyusunan konsep desain atau algoritma	Tidak ada	Tidak ada	Tidak ada
9	Penyusunan analisis data	Tidak ada	Tidak ada	Tidak ada
10	Penyusunan kesimpulan atau rekomendasi	Tidak ada	Tidak ada	Tidak ada

Bandung, 25 Mei 2026

Bryan P. Hutagalung

NIM: 18222130

ABSTRAK

Pengembangan Arsitektur Platform DataOps dan MLOps Terintegrasi pada Kubernetes dengan Pemanfaatan Open Source Tools

oleh

Bryan P. Hutagalung

18222130

Pengembangan model *machine learning* ke tahap produksi masih bersinggungan dengan fragmentasi *tool* dan pemisahan kerja antara tim data dan tim model. Praktik DataOps memperkuat kualitas dan kecepatan alur data, sementara MLOps memperkuat kehandalan siklus hidup model, namun kedua praktik tersebut sering tumbuh terpisah sehingga *lineage*, deteksi *drift*, dan penyajian fitur *batch-streaming* tidak konsisten lintas siklus. Penelitian ini mengembangkan arsitektur platform yang mengintegrasikan DataOps dan MLOps pada Kubernetes dengan memanfaatkan perangkat *open source*, sehingga satu bidang kendali menyatukan ingestasi data, pemrosesan, penyimpanan fitur, pelatihan, penyajian model, observabilitas, dan tata kelola data. Metode yang digunakan adalah *Design Science Research Methodology* (DSRM) yang melingkupi identifikasi masalah, penyusunan tujuan artefak, perancangan tujuh lapis arsitektur, implementasi berbasis GitOps, serta uji jalan menggunakan *use case* pasar aset kripto sebagai instans verifikasi. Arsitektur ini menempatkan Kubernetes sebagai bidang orkestrasi, Apache Kafka dan Apache Flink untuk *streaming*, Apache Spark untuk pemrosesan *batch*, Feast sebagai *feature store* ganda dengan ClickHouse *offline*, Valkey sebagai *online store* fitur tabular berlatensi rendah, dan Qdrant sebagai indeks vektor terpisah dengan pencarian HNSW, MLflow dan Kubeflow untuk siklus hidup model, KServe untuk penyajian, serta DataHub untuk katalog dan *lineage*. Mekanisme deteksi *drift* berbasis indikator *Population Stability Index* dan Kolmogorov-Smirnov memicu *retraining* secara otomatis melalui Argo CronWorkflow, sementara *point-in-time correctness* dijaga oleh Feast pada fase pelatihan dan penyajian *online*. Argo CD menyatukan praktik GitOps untuk reproduksibilitas *deployment* dari satu sumber kebenaran berbasis Git, dan Prometheus, Grafana, OpenTelemetry, Loki, serta Grafana Tempo menutup kebutuhan observabilitas pada tiga lapis metrik, log, dan *trace*. Hasil rancangan dan implementasi inti menunjukkan bahwa platform berhasil menyatukan komponen DataOps dan MLOps dalam satu siklus yang berkesinambungan; angka evaluasi kuantitatif akan dilengkapi pada laporan akhir setelah seluruh tahap pengujian beban dan kualitas selesai.

Kata kunci: DataOps, MLOps, Kubernetes, Feature Store, Drift Detection, GitOps, Open Source.

KATA PENGANTAR

Puji syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya, penulis dapat menyelesaikan penulisan Laporan Tugas Akhir yang berjudul “Pengembangan Arsitektur Platform DataOps dan MLOps Terintegrasi pada Kubernetes dengan Pemanfaatan *Open Source Tools*”. Laporan ini disusun sebagai salah satu syarat untuk menyelesaikan program Sarjana di Program Studi Sistem dan Teknologi Informasi, Sekolah Teknik Elektro dan Informatika, Institut Teknologi Bandung.

Penulis menyampaikan terima kasih kepada Bapak Achmad Imam Kistijantoro, S.T, M.Sc., Ph.D. selaku dosen pembimbing yang telah memberikan arahan, masukan, dan diskusi mendalam sepanjang proses pengerjaan Tugas Akhir. Penulis juga mengucapkan terima kasih kepada seluruh dosen Program Studi Sistem dan Teknologi Informasi ITB atas ilmu yang diberikan selama masa studi, serta kepada keluarga dan rekan-rekan yang senantiasa memberikan dukungan moral selama proses penulisan.

Penulis menyadari bahwa laporan ini masih memiliki kekurangan. Oleh karena itu, kritik dan saran yang membangun sangat diharapkan untuk perbaikan di masa mendatang. Semoga laporan ini bermanfaat bagi pembaca dan pengembangan platform DataOps dan MLOps di lingkungan akademik maupun industri.

Bandung, 25 Mei 2026

Penulis

Bryan P. Hutagalung

DAFTAR ISI

LEMBAR PENGESAHAN	iii
PERNYATAAN ORISINALITAS	v
PERNYATAAN PENGGUNAAN AI	vii
ABSTRAK	ix
KATA PENGANTAR	xi
DAFTAR ISI	xiii
DAFTAR GAMBAR	xvii
DAFTAR TABEL	xix
DAFTAR PERSAMAAN	xxi
DAFTAR ALGORITMA	xxiii
DAFTAR <i>LISTING</i>	xxv
DAFTAR SIMBOL	xxvii
DAFTAR SINGKATAN	xxix
I PENDAHULUAN	1
I.1 Latar Belakang	1
I.2 Rumusan Masalah	3
I.3 Tujuan	4
I.4 Batasan Masalah	5
I.5 Metodologi	6
I.6 Sistematika Penulisan	7
II STUDI LITERATUR	9
II.1 Cakupan Studi Literatur	9
II.2 DataOps dan MLOps	9
II.2.1 DataOps	9
II.2.2 MLOps	9
II.2.3 Tingkat Kematangan MLOps	10
II.2.4 Integrasi DataOps dan MLOps	13
II.3 Kubernetes dan Orkestrasi <i>Cloud-Native</i>	13
II.4 Manajemen Data: Ingestasi, <i>Streaming</i> , dan <i>Batch</i>	14
II.4.1 Pola Lambda dan Kappa	14
II.4.2 Apache Kafka	14
II.4.3 Apache Flink	14
II.4.4 Apache Spark	15
II.4.5 Format Tabel <i>Lakehouse</i>	15
II.5 Layanan Fitur (<i>Feature Store</i>)	16
II.5.1 Konsep dan Peran	16
II.5.2 Penyimpanan <i>Offline</i> dan <i>Online</i>	16
II.5.3 Pencarian Vektor dan HNSW	17
II.5.4 <i>Point-in-Time Correctness</i>	17
II.6 Siklus Hidup Model	18
II.6.1 Eksperimen dan <i>Tracking</i>	18
II.6.2 Orkestrasi Pelatihan	18

II.6.3 Penyajian Model	19
II.7 Tata Kelola Data: Katalog, <i>Lineage</i> , dan Kualitas	19
II.8 Deteksi <i>Drift</i>	20
II.9 Observabilitas <i>Cloud-Native</i>	20
II.10 GitOps dan <i>Continuous Delivery</i>	21
II.11 Keamanan Platform: Identitas, Rahasia, <i>Mesh</i> , dan Kebijakan	22
II.12 Penelitian Terkait dan Posisi Penelitian	24
III ANALISIS MASALAH	33
III.1 Analisis Kondisi Saat Ini	33
III.1.1 Fragmentasi pada Peran <i>Business User</i>	33
III.1.2 Fragmentasi pada Peran <i>Data Engineer</i>	34
III.1.3 Fragmentasi pada Peran <i>Data Scientist</i>	34
III.1.4 Fragmentasi pada Peran <i>Machine Learning Engineer</i>	35
III.1.5 Pandangan Umum Kondisi Fragmentasi	35
III.2 Analisis Kebutuhan	38
III.2.1 Identifikasi Masalah Pengguna	38
III.2.2 Kebutuhan Fungsional	38
III.2.3 Kebutuhan Nonfungsional	40
III.3 Analisis Pemilihan Solusi	42
III.3.1 Alternatif Solusi	43
III.3.2 Analisis Penentuan Solusi	43
IV PERANCANGAN ARSITEKTUR PLATFORM DATAOPS DAN MLO- PS TERINTEGRASI	47
IV.1 Gambaran Umum Platform	47
IV.2 Perancangan Arsitektur Terintegrasi	49
IV.2.1 Pandangan per Peran Pengguna	49
IV.2.2 Lapisan Infrastruktur dan Bidang Kendali	54
IV.2.3 Lapisan Ingestasi dan Pemrosesan Data	54
IV.2.4 Lapisan Siklus Hidup Model dan Penyajian	54
IV.2.5 Lapisan Tata Kelola dan Observabilitas	55
IV.3 Perancangan Sub-sistem Tata Kelola Data	55
IV.3.1 Katalog Metadata dan <i>Lineage</i>	55
IV.3.2 Kontrak Data dan Kualitas	55
IV.3.3 Identitas dan Kontrol Akses	56
IV.4 Perancangan Sub-sistem Deteksi <i>Drift</i> dan <i>Continuous Training</i>	56
IV.4.1 Pengukuran <i>Drift</i>	56
IV.4.2 Alur Pelatihan Ulang Otomatis	56
IV.4.3 Mekanisme Penanganan Kasus Khusus	57
IV.5 Perancangan Sub-sistem Layanan Fitur <i>Dual-Store</i>	57
IV.5.1 Penyimpanan <i>Offline</i> dan <i>Online</i>	57
IV.5.2 Dukungan <i>Vector Embeddings</i> dan HNSW	58
IV.5.3 Jaminan <i>Point-in-Time Correctness</i>	58
IV.5.4 Aliran Materialisasi Fitur	58
IV.6 Alur Kerja <i>End-to-End</i>	58

V	IMPLEMENTASI PLATFORM	61
V.1	Lingkungan Implementasi	61
V.1.1	Konfigurasi Dasar <i>Cluster</i>	61
V.1.2	Manajemen Rahasia dan Identitas	62
V.1.3	Pola GitOps	62
V.2	Implementasi Arsitektur Terintegrasi	62
V.2.1	Lapisan Ingestasi	62
V.2.2	Lapisan Pemrosesan	63
V.2.3	Lapisan Penyimpanan	63
V.2.4	Lapisan Siklus Hidup Model dan Penyajian	63
V.2.5	Lapisan Observabilitas	63
V.3	Implementasi Sub-sistem Tata Kelola Data	64
V.3.1	Katalog Metadata dan <i>Lineage</i>	64
V.3.2	Kontrak Data dan Kualitas	64
V.3.3	Kontrol Akses	64
V.4	Implementasi Sub-sistem Deteksi <i>Drift</i> dan <i>Continuous Training</i>	65
V.4.1	Detektor <i>Drift</i>	65
V.4.2	<i>Control Loop</i> Pelatihan Ulang	65
V.4.3	Penanganan Kasus Khusus	65
V.5	Implementasi Sub-sistem Layanan Fitur <i>Dual-Store</i>	65
V.5.1	Konfigurasi Feast	66
V.5.2	Dukungan <i>Vector Embeddings</i>	66
V.5.3	Materialisasi dan <i>Point-in-Time Correctness</i>	66
V.6	Verifikasi Implementasi	66
V.6.1	Lingkup Verifikasi	67
V.6.2	Pengamatan Hasil Awal	67
VI	EVALUASI	69
VI.1	Metode Evaluasi	69
VI.2	Hasil Evaluasi	69
VI.3	Pembahasan Hasil Evaluasi	70
VII	PENUTUP	71
VII.1	Kesimpulan	71
VII.2	Saran	71
Lampiran A	DAFTAR ARTEFAK PLATFORM	85
A.1	Tata Letak Repositori Platform	85
A.2	Tata Letak Repositori Use Case	85
A.3	Contoh Berkas Konfigurasi Feast	85
Lampiran B	HASIL VERIFIKASI USE CASE	87

DAFTAR GAMBAR

II.1	MLOps <i>level</i> 0: proses manual dengan penyerahan model satu arah.	10
II.2	MLOps <i>level</i> 1: otomasi <i>pipeline</i> pelatihan dengan komponen yang dapat dipakai ulang.	10
II.3	MLOps <i>level</i> 2: orkestrasi CI/CD/CT dengan deteksi <i>drift</i> sebagai pemicu pelatihan ulang.	11
III.1	Fragmentasi pada Peran <i>Business User</i>	34
III.2	Fragmentasi pada Peran <i>Data Engineer</i>	34
III.3	Fragmentasi pada Peran <i>Data Scientist</i>	35
III.4	Fragmentasi pada Peran <i>Machine Learning Engineer</i>	35
III.5	Pandangan Umum Fragmentasi DataOps dan MLOps	37
IV.1	Pandangan Umum Platform DataOps dan MLOps Terintegrasi	48
IV.2	Pandangan Platform Terintegrasi untuk <i>Business User</i>	50
IV.3	Pandangan Platform Terintegrasi untuk <i>Data Engineer</i>	51
IV.4	Pandangan Platform Terintegrasi untuk <i>Data Scientist</i>	52
IV.5	Pandangan Platform Terintegrasi untuk <i>Machine Learning Engineer</i>	53

DAFTAR TABEL

II.1	Perbandingan Framework Tingkat Kematangan MLOps	11
II.2	Evaluasi Alternatif <i>Schema Registry</i> untuk Apache Kafka	14
II.3	Evaluasi Alternatif <i>Framework</i> Pemrosesan Data	15
II.4	Evaluasi Alternatif Format Tabel <i>Lakehouse Open-Source</i>	16
II.5	Evaluasi Alternatif Penyimpanan <i>Offline</i> (OLAP)	16
II.6	Evaluasi Alternatif Penyimpanan <i>Online</i> (Latensi Rendah)	17
II.7	Evaluasi Alternatif <i>Vector Index Open-Source</i> untuk Pencarian Pendekatan Tetangga Terdekat	17
II.8	Evaluasi Alternatif <i>Feature Store Open-Source</i>	18
II.9	Evaluasi Alternatif <i>Experiment Tracking</i> dan <i>Model Registry</i>	18
II.10	Evaluasi Alternatif Orkestrasi ML <i>Open-Source</i>	19
II.11	Evaluasi Alternatif <i>Model Serving Open-Source</i>	19
II.12	Evaluasi Alternatif Manajemen Metadata dan <i>Governance</i>	20
II.13	Evaluasi Alternatif <i>Business Intelligence</i> dan <i>Dashboarding Open-Source</i>	21
II.14	Evaluasi Alternatif <i>Monitoring</i> dan <i>Observability</i>	21
II.15	Evaluasi Alternatif <i>GitOps Controller Open-Source</i>	22
II.16	Evaluasi Alternatif <i>Identity Provider Open-Source</i>	22
II.17	Evaluasi Alternatif Pengelola Rahasia <i>Open-Source</i>	23
II.18	Evaluasi Alternatif <i>Service Mesh Open-Source</i>	23
II.19	Evaluasi Alternatif <i>API Gateway Open-Source</i>	23
II.20	Evaluasi Alternatif <i>Policy Engine Open-Source</i>	24
II.21	Evaluasi Alternatif <i>Runtime Security Open-Source</i> pada Kubernetes	24
II.22	Perbandingan Detail Arsitektur Saat Ini dengan yang Diusulkan	25
III.1	Kebutuhan Fungsional Sistem	39
III.2	Kebutuhan Nonfungsional Sistem	41
III.3	Evaluasi Pemenuhan Kebutuhan Fungsional	44
III.4	Evaluasi Pemenuhan Kebutuhan Nonfungsional	44

DAFTAR PERSAMAAN

DAFTAR ALGORITMA

DAFTAR *LISTING*

DAFTAR SIMBOL

Notasi	Deskripsi	Pemakaian pertama kali
p99	Persentil ke-99 dari distribusi latensi permintaan; dipakai sebagai <i>indicator tail latency</i> pada SLO layanan inferensi.	Bab IV

DAFTAR SINGKATAN

Singkatan	Deskripsi	Pemakaian pertama kali
ACID	<i>Atomicity, Consistency, Isolation, Durability</i> —sifat transaksi basis data.	Bab IV
ANN	<i>Approximate Nearest Neighbor</i> —kelas algoritma pencarian tetangga terdekat secara aproksimasi pada ruang vektor.	Bab IV
API	<i>Application Programming Interface</i> .	Bab I
APISIX	Apache APISIX, <i>API gateway</i> berbasis Nginx dan etcd.	Bab IV
BERT	<i>Bidirectional Encoder Representations from Transformers</i> —model bahasa berbasis <i>Transformer</i> .	Bab IV
CD	<i>Continuous Delivery</i> atau <i>Continuous Deployment</i> .	Bab I
CI	<i>Continuous Integration</i> .	Bab I
CNPG	<i>CloudNativePG</i> , operator Kubernetes untuk PostgreSQL.	Bab IV
CPU	<i>Central Processing Unit</i> .	Bab IV
CT	<i>Continuous Training</i> —pelatihan model secara berkelanjutan dengan pemicu <i>drift</i> atau jadwal.	Bab I
DORA	<i>DevOps Research and Assessment</i> —metrik kinerja <i>delivery</i> perangkat lunak.	Bab IV
DSRM	<i>Design Science Research Methodology</i> —kerangka metodologi penelitian rekayasa artefak.	Bab I
GPU	<i>Graphics Processing Unit</i> .	Bab IV
HNSW	<i>Hierarchical Navigable Small World</i> —struktur graf untuk ANN.	Bab IV
HPA	<i>Horizontal Pod Autoscaler</i> —kontrol penskalaan jumlah <i>pod</i> pada Kubernetes.	Bab IV
HTML	<i>HyperText Markup Language</i> .	Bab V
JSON	<i>JavaScript Object Notation</i> .	Bab IV
KEDA	<i>Kubernetes Event-driven Autoscaling</i> — <i>autoscaler</i> berbasis sinyal eksternal.	Bab IV
KS	<i>Kolmogorov–Smirnov test</i> —uji statistik kesetaraan dua distribusi.	Bab IV

MLOps	<i>Machine Learning Operations</i> —disiplin praktik operasional pengembangan dan penyajian model <i>machine learning</i> .	Bab I
ONNX	<i>Open Neural Network Exchange</i> —format pertukaran model lintas <i>framework</i> .	Bab IV
OIDC	<i>OpenID Connect</i> —protokol identitas berbasis OAuth 2.0.	Bab IV
PSI	<i>Population Stability Index</i> —metrik pergeseran distribusi fitur antar dua periode.	Bab IV
REST	<i>Representational State Transfer</i> .	Bab IV
SDK	<i>Software Development Kit</i> .	Bab IV
SLA	<i>Service Level Agreement</i> .	Bab IV
SLO	<i>Service Level Objective</i> .	Bab IV
TLS	<i>Transport Layer Security</i> .	Bab IV
VPA	<i>Vertical Pod Autoscaler</i> —kontrol penyetelan <i>request/limit</i> sumber daya <i>pod</i> .	Bab IV

BAB I

PENDAHULUAN

I.1 Latar Belakang

Praktik *machine learning* pada lingkungan produksi tidak lagi diukur dari ketepatan model semata, melainkan dari kemampuan organisasi memindahkan model dari laptop *data scientist* ke layanan yang berjalan stabil, terpantau, dan dapat diperbarui ketika data berubah. Sculley dkk. (2015) menunjukkan bahwa kode model hanya berperan sebagai bagian kecil dari sistem *machine learning* di lapangan; sisanya berupa pengelolaan data, konfigurasi, pemantauan, layanan, dan perekaman *lineage*. Paleyes, Urma, dan Lawrence (2022) memperkuat temuan tersebut dengan memetakan tantangan utama yang muncul ketika model dipasang di produksi, dari persoalan kualitas data hingga pemeliharaan model setelah penyebaran.

Praktik DevOps yang diadaptasi oleh komunitas *machine learning* kemudian dikenal sebagai MLOps. Kreuzberger, Kühl, dan Hirschl (2023) mendefinisikan MLOps sebagai disiplin yang menyatukan rekayasa perangkat lunak, operasional, dan ilmu data untuk mengotomasi siklus hidup model dari eksperimen ke produksi. Pada saat yang sama, DataOps tumbuh sebagai pendekatan serupa untuk *pipeline* data: penekanan pada kolaborasi antar peran, kualitas data, dan iterasi cepat ditarik dari prinsip DevOps dan diterapkan pada penyiapan data. Rella (2022) mendokumentasikan bahwa MLOps dan DataOps secara konseptual sejalan, tetapi implementasinya kerap terpecah menjadi dua tumpukan teknologi yang berbeda. Akibatnya, model yang membutuhkan data segar dan terkurasi sering kali menerima data yang berasal dari *pipeline* yang tidak sinkron dengan *pipeline* pelatihan; perubahan skema di sisi data tidak terbaca otomatis di sisi model; dan kegagalan kualitas data di *batch* subuh tidak menjadi sinyal yang memicu pemeriksaan ulang model yang sudah dilayani.

Pemicu pertama pengembangan Tugas Akhir ini bersumber dari fragmentasi *tool*. Survei pemetaan arsitektur MLOps oleh Amou Najafabadi dkk. (2024) menemukan lebih dari empat puluh *tool* yang biasa dirangkai untuk mendukung satu platform produksi. Organisasi yang berinvestasi pada platform dalam negeri sering menyu-

sun rangkaian *tool* yang serupa, namun karena tiap *tool* berdiri sendiri-sendiri, integrasi antar komponen menjadi pekerjaan rekayasa berulang. Burgueno-Romero, Cánovas Izquierdo, dan García-García (2025) menyoroti bahwa biaya integrasi pada arsitektur MLOps *open source* dapat menyamai biaya pengembangan model itu sendiri, terlebih ketika tim juga harus menjaga konsistensi versi pustaka dan kontrak data lintas komponen. Jain dan Das (2025) secara eksplisit menempatkan integrasi DataOps dan MLOps sebagai prasyarat skalabilitas, namun belum banyak arsitektur referensi yang menyatukan keduanya dalam satu bidang kendali pada Kubernetes.

Pemicu kedua menyentuh tata kelola data. Bernardo dkk. (2024) memetakan tata kelola sebagai gabungan katalog, kualitas, kepemilikan, dan keamanan; semuanya membutuhkan ditegakkannya kontrak antara penghasil data dan konsumen data. Tata kelola yang baik di sisi data tidak otomatis terbawa ke sisi model: ketika sebuah *feature* ditulis ulang oleh tim data, tim model perlu mengetahui perubahan tersebut sebelum melakukan *retraining*, dan ketika model dihentikan layanannya, lineage data harus mencerminkan bahwa konsumen tersebut sudah tidak ada. Stoyanovich, Howe, dan Jagadish (2020) menggarisbawahi bahwa tanggung jawab atas data tidak bisa berhenti pada lapis penyimpanan saja, melainkan harus mengikuti seluruh siklus dari produksi data hingga prediksi yang dilayani. Tanpa platform yang menempatkan tata kelola sebagai komponen kelas pertama, *lineage* antara data, fitur, dan model menjadi terputus dan kepatuhan terhadap kebijakan akses sulit dibuktikan.

Pemicu ketiga adalah *drift* dan kebocoran temporal. Vela dkk. (2022) memperlihatkan bahwa degradasi performa model lintas waktu adalah hal yang umum, bukan kasus istimewa. Lu dkk. (2019) memberikan tinjauan luas tentang *concept drift*, dan Bayram, Ahmed, dan Kassler (2022) memetakan keluarga detektor performansi yang dapat dipasang pada *pipeline* produksi. Namun, mendeteksi *drift* tidak cukup; *pipeline* pelatihan juga harus terbebas dari kebocoran temporal yang muncul ketika fitur dengan referensi waktu masa depan tercampur ke dalam kumpulan data pelatihan masa lalu. Wang dan Ruf (2021) mengilustrasikan dampak kebocoran temporal pada *backtesting* keuangan, dan implikasinya berlaku umum: setiap domain yang menggabungkan data *batch* dengan data *streaming* berisiko menghasilkan model yang “bocor” karena ketidakkonsistenan referensi waktu antara saat pelatihan dan saat penyajian. Tanpa *point-in-time correctness* yang ditegakkan oleh layanan fitur, kesalahan ini hampir mustahil dideteksi dari log model saja.

Pemicu keempat berkaitan dengan fitur multimoda. Fitur tabular klasik bertumpang

tindih dengan fitur agregat *rolling window* dan dengan fitur *embedding* berdimensi tinggi yang dihasilkan oleh model bahasa seperti BERT (Devlin dkk. 2019) atau model *embedding* OpenAI (OpenAI December 2022). Pencarian terdekat untuk *embedding* memerlukan struktur indeks khusus, dan Malkov dan Yashunin (2020) menempatkan *Hierarchical Navigable Small World* (HNSW) sebagai opsi tepat-guna untuk pencarian pendekatan tetangga terdekat. Saat fitur tabular, fitur *rolling*, dan fitur vektor dilayani dengan kontrak yang berbeda-beda, biaya pemeliharaan kontrak *pipeline* meningkat tajam dan layanan model rawan tidak konsisten antara saat pelatihan dan saat penyajian *online*. Lamer dkk. (2024) dan Li dkk. (2023) menunjukkan bahwa *feature store* berperan sebagai jembatan kontrak antara dua mode tersebut; tetapi sebagian besar implementasi *open source* masih membutuhkan integrasi tangan dengan basis data dan sistem pencarian vektor secara terpisah.

Empat tekanan tersebut menggerakkan pengembangan sebuah platform yang menyatukan praktik DataOps dan MLOps di atas Kubernetes dengan komponen *open source*. Kubernetes dipilih karena perannya sebagai standar *de facto* bagi orkestrasi beban kerja *cloud-native* dan karena adopsi luasnya pada ekosistem *tool* yang dibutuhkan, sebagaimana didokumentasikan oleh Lakka (2025). Pemanfaatan komponen *open source* dipilih agar arsitektur dapat ditiru tanpa keterikatan pada satu penyedia layanan, sekaligus mempermudah audit teknis. Rancangan platform sengaja dijaga *domain-agnostic* agar dapat diadaptasi pada beragam domain tanpa perubahan pada lapis kendali.

I.2 Rumusan Masalah

Latar belakang di atas mengarah pada empat rumusan masalah yang ingin diselesaikan oleh platform yang dikembangkan. Keempat butir di bawah ini selanjutnya dirujuk dengan singkatan RM-1 hingga RM-4 pada bab-bab berikutnya.

1. Fragmentasi tumpukan teknologi DataOps dan MLOps

Tool pada lapis ingestasi, pemrosesan, penyimpanan, pelatihan, dan penyajian model masih dirakit secara terpisah sehingga integrasi memerlukan biaya rekayasa berulang dan menimbulkan duplikasi konsep antara *pipeline* data dan *pipeline* model. Persoalan ini berakibat pada lemahnya konsistensi kontrak data antara fase pelatihan dan fase penyajian, serta tingginya beban pemeliharaan ketika ada perubahan versi salah satu komponen.

2. Tata kelola data yang tidak konsisten lintas siklus data–fitur–model

Katalog metadata, *lineage*, dan kontrol kualitas sering dijalankan sebagai proses pasca-fakta, terpisah dari *pipeline* produksi data dan dari pencatatan eks-

perimen model. Akibatnya, ketika sebuah fitur diperbarui atau sebuah model dihentikan, perubahan tersebut tidak otomatis tercermin pada katalog dan riwayat kepemilikan, sehingga sulit memenuhi kewajiban audit dan reproduktibilitas.

3. Deteksi *drift* yang tidak terotomasi dan risiko kebocoran temporal antara mode *batch* dan *streaming*

Tanda-tanda perubahan distribusi data, perubahan hubungan masukan–keluaran, dan pencampuran referensi waktu antara fitur *streaming* dengan fitur *batch* berpotensi mengempaskan kualitas model di lapangan tanpa terdeteksi pemilik model. Mekanisme deteksi *drift* yang aktif dan jaminan *point-in-time correctness* pada penyajian fitur belum tertanam sebagai bagian standar dari *pipeline* produksi.

4. Penyajian fitur multimoda dari satu sumber kebenaran

Fitur tabular, fitur agregat, dan fitur *embedding* vektor berdimensi tinggi memiliki pola akses dan SLA berbeda, namun tetap harus konsisten antara nilai yang dipakai saat pelatihan dan nilai yang dilayani saat *inference*. Tanpa layanan fitur yang menyatukan jalur *offline* dan *online* pada satu kontrak API, biaya pemeliharaan kontrak meningkat dan risiko *train–serving skew* membesar.

I.3 Tujuan

Penelitian ini merumuskan empat tujuan yang masing-masing menjawab satu rumusan masalah, dengan luaran berupa artefak teknis yang dapat ditunjukkan dan dipakai ulang. Keempat tujuan di bawah ini selanjutnya dirujuk dengan singkatan T-1 hingga T-4 pada bab-bab berikutnya.

1. Arsitektur platform DataOps dan MLOps terintegrasi

Merancang dan mewujudkan arsitektur platform DataOps dan MLOps terintegrasi di atas Kubernetes yang menyatukan lapis ingestasi, pemrosesan, penyimpanan fitur, pelatihan, penyajian model, observabilitas, dan tata kelola pada satu bidang kendali deklaratif dengan praktik GitOps.

2. Sub-sistem tata kelola data

Membangun *sub-sistem* tata kelola data yang menyediakan katalog metadata, perekaman *lineage*, dan kontrol kualitas data sebagai layanan bersama yang otomatis dijalankan di seluruh *pipeline* data, fitur, dan model.

3. Sub-sistem deteksi *drift* terotomasi

Mengimplementasikan *sub-sistem* deteksi *drift* terotomasi dengan kebijakan

retraining berbasis ambang batas serta menjamin *point-in-time correctness* pada penyajian fitur lintas mode *batch* dan *streaming*.

4. Layanan fitur *dual-store offline-online*

Mengembangkan layanan fitur *dual-store offline-online* yang melayani fitur tabular, fitur agregat, dan fitur vektor berdimensi tinggi melalui satu kontrak API dengan jaminan konsistensi nilai antara fase pelatihan dan fase penyajian. Kriteria keberhasilan dari keseluruhan platform diukur dari tiga sudut: pertama, seluruh artefak dapat dipasang dari satu repositori Git melalui Argo CD tanpa intervensi manual; kedua, seluruh *sub-sistem* berkomunikasi melalui kontrak yang ditekankan oleh layanan tata kelola dan layanan fitur; ketiga, mekanisme deteksi *drift* memicu *retraining* secara otomatis ketika ambang batas dilewati pada *use case* verifikasi.

I.4 Batasan Masalah

Pengembangan platform pada Tugas Akhir ini dibatasi sebagai berikut. Kelima batasan di bawah ini selanjutnya dirujuk dengan singkatan B-1 hingga B-5 pada bab-bab berikutnya.

1. Fokus pada lapis kendali platform

Fokus utama berada pada lapis kendali platform, bukan pada pengembangan model spesifik. *Use case* domain hanya dipakai sebagai instans verifikasi rancangan; karena itu pembahasan masalah, perancangan, dan implementasi platform dijaga *domain-agnostic*.

2. Lingkungan Kubernetes *k3s* satu *node*

Platform berjalan di atas Kubernetes dengan distribusi *k3s* pada satu *node* sebagai lingkungan pengembangan dan pengujian. Setiap beban kerja diatur minimal satu replika ketika *idle* dan diskalakan oleh *Horizontal Pod Autoscaler* (HPA), *Vertical Pod Autoscaler* (VPA), atau *Kubernetes Event-driven Autoscaling* (KEDA) ketika beban nyata muncul.

3. Komponen *open source* dengan patokan versi April 2026

Seluruh komponen yang digunakan berbasis *open source*. Versi pustaka dan basis *image container* dipatok pada rilis stabil tertinggi per April 2026.

4. Cakupan keamanan terbatas pada akses antar layanan internal

Otentikasi *end-user* aplikasi domain tidak menjadi bagian inti rancangan; aspek keamanan yang dibahas berfokus pada akses antar layanan internal melalui *mutual TLS*, manajemen rahasia melalui OpenBao, dan kebijakan jaringan berbasis Kubernetes.

5. Evaluasi kuantitatif menyusul setelah implementasi inti

Evaluasi *end-to-end* seperti pengukuran latensi penuh, *throughput*, dan stabilitas penyebaran lanjutan dilakukan setelah seluruh tahapan implementasi inti berjalan; angka kuantitatif evaluasi dilaporkan pada bagian Evaluasi setelah pengujian lengkap.

I.5 Metodologi

Penelitian ini mengikuti kerangka *Design Science Research Methodology* (DSRM) yang diperkenalkan oleh Peffers dkk. (2007) dan diadaptasi untuk pengembangan platform perangkat lunak. Pemilihan DSRM mengikuti pertimbangan bahwa luaran utama Tugas Akhir berupa artefak teknis berupa rancangan dan implementasi platform, sementara penilaian artefak dilakukan dengan demonstrasi pada *use case* verifikasi—pola yang persis dianjurkan oleh DSRM untuk penelitian yang menghasilkan artefak. Tahapan yang dijalankan meliputi enam fase berikut.

1. Identifikasi masalah dan motivasi

Studi literatur mengenai praktik DataOps, MLOps, dan tata kelola data dilakukan untuk menyusun peta tantangan. Pengamatan pada studi kasus produksi yang terdokumentasi dipakai untuk mempertajam keempat rumusan masalah.

2. Penetapan tujuan artefak

Tujuan dirumuskan sebagai daftar artefak teknis yang harus dapat dipasang, dioperasikan, dan diaudit. Tujuan dipetakan satu lawan satu terhadap rumusan masalah untuk menghindari ketidaksinkronan antara persoalan dan capaian.

3. Perancangan dan pengembangan artefak

Arsitektur tujuh lapis dirancang sebagai bidang kendali deklaratif berbasis Kubernetes. Untuk setiap lapis dilakukan pemilihan komponen *open source* berdasarkan kriteria fungsi, dukungan ekosistem, kompatibilitas versi April 2026, dan kemampuan diorkestrasi oleh Argo CD dengan praktik GitOps.

4. Demonstrasi

Platform dipasang pada lingkungan *k3s* dan diberi muatan dari *use case* pasar aset kripto sebagai instans verifikasi. Aliran data *streaming*, transformasi fitur, pelatihan model, penyajian model, dan deteksi *drift* dijalankan dalam satu lingkaran tertutup.

Use case ini dipilih sebagai instans verifikasi karena pasar aset kripto beroperasi 24 jam sehari dengan volatilitas harga yang tinggi, sehingga menghasilkan aliran *event streaming* yang kontinu, distribusi fitur yang berubah cepat, dan kebutuhan fitur multimoda (tabular, agregat *rolling window*, dan *embe*

dding vektor). Kondisi tersebut menguji jalur ingestasi, materialisasi fitur, dan deteksi *drift* secara *end-to-end* tanpa menjadikan domain kripto sebagai fokus penelitian.

5. **Evaluasi**

Evaluasi dilaksanakan pada akhir siklus implementasi dengan tiga dimensi: fungsional (apakah semua kebutuhan fungsional berjalan), nonfungsional (skalabilitas, ketersediaan, *observability*, keamanan, dan biaya), serta tata kelola (*lineage* tertelusur, kontrak data ditegakkan, kebijakan akses dapat diaudit). Angka kuantitatif dilaporkan pada Bab Evaluasi.

6. **Komunikasi**

Hasil rancangan dan implementasi didokumentasikan pada laporan Tugas Akhir ini, beserta repositori sumber yang menjadi acuan reproduksi.

I.6 Sistematika Penulisan

Laporan Tugas Akhir disusun atas tujuh bab dengan urutan sebagai berikut.

1. **Bab I Pendahuluan**

Bab ini memuat latar belakang, rumusan masalah, tujuan, batasan masalah, metodologi penelitian, serta sistematika penulisan.

2. **Bab II Studi Literatur**

Bab ini meninjau konsep DataOps, MLOps, Kubernetes, manajemen data berbasis Kafka–Flink–Spark, *feature store*, siklus hidup model dengan MLflow dan Kubeflow, penyajian model dengan KServe, tata kelola data, deteksi *drift*, observabilitas *cloud-native*, dan praktik GitOps. Tinjauan ditutup dengan pemetaan penelitian terkait yang menjadi posisi platform yang dikembangkan.

3. **Bab III Analisis**

Bab ini memetakan kondisi sistem yang ada saat ini, menyusun kebutuhan fungsional dan nonfungsional, membandingkan alternatif solusi per lapis arsitektur, dan menutup dengan pemilihan komponen yang dipakai oleh platform.

4. **Bab IV Perancangan Arsitektur Platform DataOps dan MLOps Terintegrasi**

Bab ini menjabarkan gambaran umum arsitektur, perancangan tiap *sub-sistem* yang menjawab keempat tujuan, dan alur kerja *end-to-end* yang menghubungkan komponen.

5. **Bab V Implementasi Platform**

Bab ini memaparkan implementasi tiap *sub-sistem* yang sejalan dengan Bab

IV, lengkap dengan praktik GitOps, manajemen rahasia, kebijakan jaringan, dan otomasi *continuous training*.

6. Bab VI Evaluasi

Bab ini berisi metode evaluasi, hasil pengukuran, dan pembahasan hasil terhadap kebutuhan fungsional dan nonfungsional yang ditetapkan pada Bab III.

7. Bab VII Kesimpulan dan Saran

Bab ini menyimpulkan jawaban atas keempat tujuan dan memberikan saran pengembangan lanjutan.

Setelah Bab VII, dokumen ditutup oleh Daftar Pustaka yang memuat seluruh referensi terkutip dan Lampiran yang memuat berkas pendukung seperti manifes Kubernetes, kontrak data, serta tangkapan layar antarmuka observabilitas.

BAB II

STUDI LITERATUR

II.1 Cakupan Studi Literatur

Bab ini merangkum landasan teori dan ekosistem perangkat lunak yang relevan untuk perancangan platform pada Bab IV. Tinjauan disusun mulai dari konsep DataOps dan MLOps, dilanjutkan dengan komponen teknis yang menjadi bahan baku perancangan, kemudian ditutup dengan posisi penelitian terdahulu dan kebaruan yang ditawarkan.

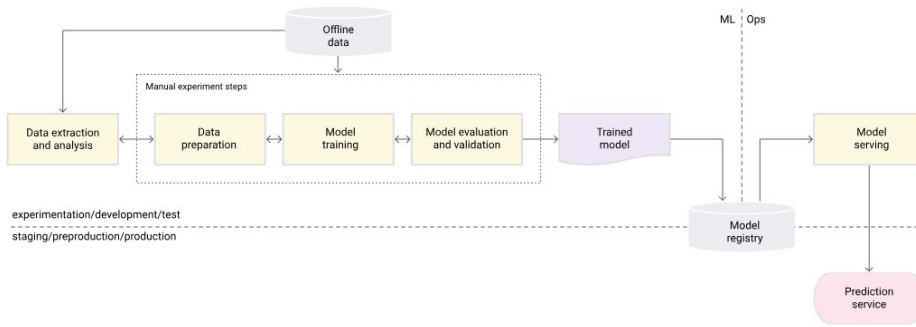
II.2 DataOps dan MLOps

II.2.1 DataOps

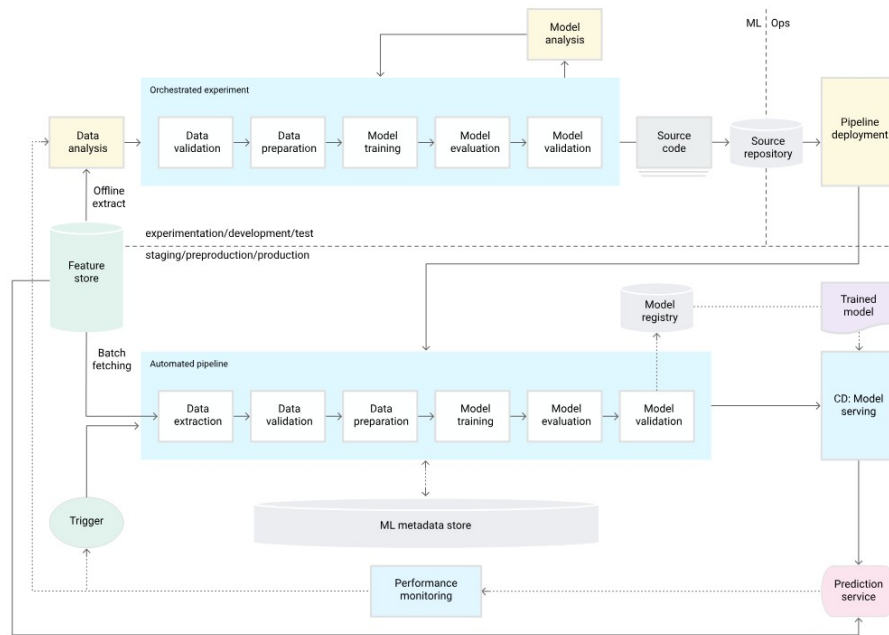
DataOps merupakan adaptasi praktik DevOps untuk siklus hidup data. Rella (2022) mendefinisikan DataOps sebagai pendekatan kolaboratif yang menempatkan kualitas data, kecepatan iterasi, dan pengukuran tertaut sebagai inti. Kleppmann (2017) memberi fondasi konseptual lewat penjelasan tentang sistem berbasis data yang andal, dapat diskalakan, dan dapat dipelihara, sementara Stopford (2018) memperkenalkan pola arsitektur *event-driven* dengan Apache Kafka sebagai tulang punggung. Pada praktiknya, DataOps menuntut otomatisasi *pipeline* ingestasi dan transformasi, pengujian kontrak data, perekaman *lineage*, dan pemantauan kualitas data sebagai bagian permanen dari operasi.

II.2.2 MLOps

MLOps memperluas DevOps ke seluruh siklus hidup model *machine learning*. Kreuzberger, Kühl, dan Hirschl (2023) memberikan definisi konsolidatif: MLOps sebagai gabungan budaya kerja dan perangkat lunak untuk mengotomasi eksperimen, pelatihan, evaluasi, penyebaran, dan pemantauan model. Sculley dkk. (2015) menjelaskan akar persoalannya: *hidden technical debt* pada sistem *machine learning* muncul terutama dari komponen non-model seperti perekayasa fitur, konfigurasi, perekaman, pengujian data, dan layanan pemantauan. Paleyes, Urma, dan Lawrence (2022) melengkapi pemetaan dengan survei tantangan operasionalisasi, dan Shankar dkk. (2022) membawa perspektif praktisi melalui wawancara dengan tim MLOps lintas



Gambar II.1 MLOps *level 0*: proses manual dengan penyerahan model satu arah.



Gambar II.2 MLOps *level 1*: otomasi *pipeline* pelatihan dengan komponen yang dapat dipakai ulang.

industri.

II.2.3 Tingkat Kematangan MLOps

Beberapa kerangka kematangan diusulkan untuk menggambarkan jenjang kapabilitas tim. Google Cloud (2024) membagi MLOps menjadi tiga level: *level 0* dengan proses manual, *level 1* dengan otomasi *pipeline* pelatihan, dan *level 2* dengan otomasi penyebaran berbasis CI/CD/CT. Tiga level ini diilustrasikan pada Gambar II.1, Gambar II.2, dan Gambar II.3. Microsoft Azure (2023) memberi varian dengan lima level (0–4), dan McKnight (January 2020) menambah perspektif organisasi. Perbandingan ringkas dapat dilihat pada Tabel II.1.

FW	L	DevOps Practices	ML Pipeline Automation	CI/CD Integration	CT (Continuous Training)	Monitoring & Feedback Loop	Governance & Lineage
	L1	Ya (aplikasi saja)	Manual (<i>model handoff</i>)	Aplikasi saja (bukan untuk ML)	Tidak ada	Minimal (<i>limited feedback</i>)	Minimal (sulit ditelusuri)
	L2	Ya (DevOps aplikasi)	Ya (pelatihan saja)	Tidak ada (<i>release manual</i>)	Ya (pelatihan otomatis)	Parsial (pelacakan terpusat)	Ya (<i>version control</i> , manajemen model)
	L3	Ya (terintegrasi)	Ya (latih dan <i>deploy</i>)	Ya (CD model)	Ya (pelatihan otomatis)	Parsial (<i>A/B testing</i>)	Ya (pengujian terintegrasi)
	L4	Ya (<i>fully integrated</i>)	Ya (<i>fully automated</i>)	Ya (CI-/CD/CT penuh)	Ya (<i>auto retrain</i>)	Ya (<i>verbose metrics, closed loop</i>)	Ya (komprehensif)
AWS	Initial	Tidak ada	Manual (<i>notebooks</i> , lokal)	Tidak ada	Tidak ada	Tidak ada	Tidak ada (<i>no standardization</i>)
	Repeatable	Parsial (infrastruktur)	Ya (<i>training pipeline</i>)	Tidak ada (<i>deploy manual</i>)	Ya (<i>automated</i>)	Minimal (infrastruktur eksperimen)	Ya (<i>model registry, version control</i>)
	Reliable	Ya (terintegrasi)	Ya (<i>automated</i>)	Ya (CI/CD terintegrasi)	Ya (<i>automated</i>)	Ya (<i>prod monitoring, rollback</i>)	Ya (IaC, pengujian otomatis)
	Scalable	Ya (multi-account/region)	Ya (<i>fully automated, multi-region</i>)	Ya (otomasi penuh)	Ya (<i>auto retrain melalui drift detection</i>)	Ya (<i>drift detection, optimasi otomatis</i>)	Ya (<i>centralized feature store, full governance</i>)
Giga Om	L0	Tidak ada (<i>no standards</i>)	Manual (inkonsisten)	Tidak ada	Tidak ada	Minimal	Minimal (keamanan minimal)
	L1	Parsial (<i>basic practices</i>)	Parsial (sebagian otomatis)	Tidak ada (sebagian besar manual)	Tidak ada	Minimal (<i>basic tracking</i>)	Parsial (<i>limited governance</i>)

FW	L	DevOps Practices	ML Pipeline Automation	CI/CD Integration	CT (Continuous Training)	Monitoring & Feedback Loop	Governance & Lineage
	L2	Ya (diimplementasikan)	Ya (pipeline CI/CD)	Ya (diimplementasikan)	Parsial (sebagian otomatis)	Parsial (basic monitoring)	Ya (formal governance dimulai)
	L3	Ya (fully integrated)	Ya (fully automated)	Ya (komprehensif)	Ya (automated)	Ya (comprehensive monitoring/alerting)	Ya (automated governance, self-service)
	L4	Ya (fully adaptive)	Ya (AI-optimized)	Ya (self-optimizing)	Ya (continuous learning)	Ya (continuous learning dari produksi)	Ya (platform terintegrasi dan adaptif penuh)

Keterangan:

1. Merah : Tidak ada implementasi atau sepenuhnya manual
2. Kuning : Implementasi parsial atau terbatas
3. Hijau : Implementasi penuh atau otomatis

II.2.4 Integrasi DataOps dan MLOps

Jain dan Das (2025) menyatakan bahwa integrasi DataOps dan MLOps menjadi prasyarat skalabilitas dan ketahanan *pipeline machine learning* modern. Burgueno-Romero, Cánovas Izquierdo, dan García-García (2025) mengusulkan arsitektur *open source* yang menyatukan kedua praktik tersebut, namun belum membahas penyajian fitur *dual-store* dan kebijakan deteksi *drift* sebagai satu kesatuan. Rodrigues dkk. (2025) membahas arsitektur untuk *stream learning* mendekati waktu nyata, dengan penekanan pada deteksi *drift* dan pembaruan model. Penelitian ini berdiri pada irisan ketiganya: arsitektur *open source* yang menyatukan DataOps dan MLOps, dengan layanan fitur *dual-store* dan deteksi *drift* terotomasi sebagai kelas pertama.

II.3 Kubernetes dan Orkestrasi Cloud-Native

Kubernetes berperan sebagai bidang orkestrasi yang menjalankan beban kerja terkontainerisasi pada *cluster* mesin. Lakka (2025) mendokumentasikan adopsi Kubernetes pada skala perusahaan dan menyoroti perannya sebagai standar *de facto cloud-native*. Adusumilli (2025) memperluas perspektif tersebut dengan pola *serverless* di atas Kubernetes, yang relevan untuk beban kerja *inference* dengan pola lalu lintas berfluktuasi.

Pemilihan Kubernetes sebagai bidang dasar memberi tiga keuntungan teknis: orkestrasi beban kerja melalui objek deklaratif, mekanisme penskalaan otomatis lewat HPA, VPA, dan KEDA, serta ekosistem operator yang menyediakan perangkat seperti *operator* Kafka, Spark, Flink, MLflow, dan Argo CD. Distribusi *lightweight* seperti *k3s* mengakomodasi pengembangan dan pengujian pada satu *node* tanpa mengorbankan kompatibilitas API.

II.4 Manajemen Data: Ingestasi, *Streaming*, dan *Batch*

II.4.1 Pola Lambda dan Kappa

Marz dan Warren (2015) memperkenalkan arsitektur Lambda yang memisahkan *batch layer* dan *speed layer*. Kreps (2014) mengkritisi pola tersebut karena ganda operasi dan mengusulkan arsitektur Kappa yang menyatukan keduanya menjadi satu jalur *stream-first*. Amazon Web Services (2024) memberikan rangkuman pola *event-driven* yang banyak diadopsi di lingkungan *cloud-native*.

II.4.2 Apache Kafka

Apache Kafka berperan sebagai *distributed log* yang menyimpan kejadian dalam urutan tertentu dan memungkinkan pelanggan untuk memutar ulang aliran data dari posisi tertentu (Apache Software Foundation 2024c). Kafka mendukung jaminan *at-least-once* secara baku dan *exactly-once* dengan transaksi, sehingga cocok untuk dipakai sebagai tulang punggung penghubung antara produsen data dan konsumen pemroses. Registrasi skema dipisahkan ke Karapace (Aiven Karapace Authors 2025) sebagai *schema registry* kompatibel-Confluent yang berlisensi Apache 2.0, dengan dukungan Avro, JSON Schema, dan Protobuf serta verifikasi kompatibilitas saat publikasi. Tabel II.2 merangkum perbandingan beberapa pilihan *schema registry open-source*.

Tabel II.2 Evaluasi Alternatif *Schema Registry* untuk Apache Kafka

<i>Schema Registry</i>	Lisensi <i>Open</i>	Kompatibilitas Confluent	Dukungan Format	Maturitas	Total
Karapace (Aiven)	5	5	5	4	19
Apicurio Registry	5	4	5	4	18
Confluent <i>Schema Registry</i>	2	5	5	5	17
Strimzi <i>Schema Registry</i>	5	3	3	3	14

II.4.3 Apache Flink

Apache Flink memproses aliran data dengan dukungan *event-time* dan *stateful operations*. Carbone dkk. (2015) menjelaskan dasar *pipeline streaming* pada Flink, dan

dokumentasi resmi (Apache Software Foundation 2024b) menjabarkan fitur *exactly-once processing* dengan *checkpointing*. Flink dipakai untuk transformasi fitur waktu nyata dan agregasi *rolling window* yang dibutuhkan oleh model *streaming*.

II.4.4 Apache Spark

Apache Spark menyediakan kemampuan pemrosesan *batch* dan *micro-batch* pada skala besar (Apache Software Foundation 2024d). Zaharia dkk. (2016) memberikan ringkasan arsitektur Spark sebagai mesin terpadu. Spark dipakai untuk transformasi fitur dengan referensi waktu historis dan untuk pelatihan model yang membutuhkan pemrosesan data besar. Transformasi SQL deklaratif pada lapisan *warehouse* dikerjakan oleh dbt (dbt Labs 2025), yang memetakan model ke berkas SQL dengan dukungan pengujian skema dan dokumentasi otomatis.

Tabel II.3 Evaluasi Alternatif *Framework* Pemrosesan Data

<i>Framework Pemrosesan</i>	Maturitas	Kapabilitas <i>Batch</i>	Kapabilitas <i>Streaming</i>	Komunitas	Total
Apache Spark	5	5	4	5	19
Apache Flink	4	4	5	4	17
Apache Kafka + Streams	5	2	5	4	16
Apache Beam	3	4	4	3	14

II.4.5 Format Tabel *Lakehouse*

Penyimpanan jangka panjang pada *data lake* memerlukan format tabel yang menyediakan transaksi ACID, evolusi skema yang aman, dan ketertelusuran lintas *engine* pemrosesan. Apache Iceberg, Delta Lake, dan Apache Hudi merupakan tiga format tabel *open-source* yang lazim dipakai untuk arsitektur *lakehouse*, sementara penyimpanan langsung dalam berkas Parquet tanpa lapisan format tabel tidak menjamin transaksi maupun evolusi skema yang aman. Pada platform ini, Apache Iceberg dipilih sebagai format tabel *lakehouse* karena netralitas *engine*, dukungan katalog REST melalui Lakekeeper (Lakekeeper Authors 2025), dan kemampuan kueri federasi melalui Trino (Trino Software Foundation 2024). Tabel II.4 merangkum perbandingan beberapa format tabel *lakehouse* terhadap kriteria yang relevan dengan platform.

Tabel II.4 Evaluasi Alternatif Format Tabel *Lakehouse Open-Source*

Format Tabel	Maturitas	Transaksi ACID	Evolusi Skema	Integrasi Engine	Total
Apache Iceberg	5	5	5	5	20
Delta Lake	5	5	4	4	18
Apache Hudi	4	5	4	3	16
Parquet (tanpa format tabel)	5	1	2	3	11

II.5 Layanan Fitur (*Feature Store*)

II.5.1 Konsep dan Peran

Feature store berperan sebagai lapis penyatu antara produksi fitur dan konsumsi fitur. Lamer dkk. (2024) menjelaskan posisi *feature store* di antara *data lake*, *data warehouse*, dan *datamart*, sebagai komponen yang menyiapkan fitur untuk konsumsi model. Li dkk. (2023) menjabarkan arsitektur *feature store geo-distributed* dan menekankan pentingnya kontrak fitur yang seragam antara *batch* dan *online*.

II.5.2 Penyimpanan *Offline* dan *Online*

Penyimpanan *offline* dipakai untuk pelatihan dan *batch scoring*; penyimpanan *online* dipakai untuk *inference* dengan latensi rendah. Tabel II.5 dan Tabel II.6 membandingkan beberapa pilihan teknologi untuk kedua peran tersebut. Feast (Feast Project 2024) dipilih sebagai kerangka *feature store* karena modularitas penyimpanan dan kontrak API yang konsisten. ClickHouse (ClickHouse Inc 2024) dipilih sebagai *offline store* karena kompresi kolumnar dan latensi kueri agregasi yang rendah, sedangkan Valkey (Valkey Contributors 2025) dipilih sebagai *online store* karena kompatibilitas wire-protocol RESP dengan ekosistem klien Redis dan lisensi BSD yang tetap terbuka pasca-perubahan lisensi Redis.

Tabel II.5 Evaluasi Alternatif Penyimpanan *Offline* (OLAP)

Penyimpanan <i>Offline</i>	Maturitas	Performa Query	Dukungan Ingestion	Komunitas	Total
ClickHouse	5	5	4	4	18
Apache Pinot	4	4	5	3	16
Apache Druid	4	4	5	4	17
Apache Kudu	3	3	4	3	13

Tabel II.6 Evaluasi Alternatif Penyimpanan *Online* (Latensi Rendah)

Penyimpanan <i>Online</i>	Maturitas	Latensi	Throughput	Lisensi <i>Open</i>	Total
Valkey (RESP)	5	5	5	5	20
Redis <i>Community</i>	5	5	5	3	18
DragonflyDB	4	5	5	3	17
Apache Cassandra	5	4	5	5	19

II.5.3 Pencarian Vektor dan HNSW

Fitur *embedding* berdimensi tinggi memerlukan struktur indeks pendekatan tetangga terdekat (*approximate nearest neighbor*, ANN). Malkov dan Yashunin (2020) memperkenalkan *Hierarchical Navigable Small World* (HNSW) sebagai struktur indeks dengan keseimbangan baik antara kecepatan kueri dan kualitas hasil. Alternatif lain seperti *Product Quantization* (Jegou, Douze, dan Schmid 2011) dan pencarian skala miliar dengan GPU (Johnson, Douze, dan Jégou 2021) relevan untuk skala yang berbeda. Untuk platform ini, Qdrant (Qdrant Team 2025) dipilih sebagai *vector index* terpisah dari *online store* tabular karena dukungan HNSW *native*, mode *gRPC* berlatensi rendah, dan kemampuan filter pada *payload* yang menyatu dengan kueri kemiripan. Pemisahan ini menjaga jalur fitur tabular pada protokol RESP tetap ringan, sedangkan pencarian vektor mendapat *engine* yang dirancang khusus untuk dimensi tinggi. Tabel II.7 merangkum perbandingan beberapa pilihan *vector index open-source* pada empat kriteria yang relevan dengan kebutuhan platform.

Tabel II.7 Evaluasi Alternatif *Vector Index Open-Source* untuk Pencarian Pendekatan Tetangga Terdekat

<i>Vector Index</i>	Maturitas	Latensi Rendah	Komunitas	Operabilitas K8s	Total
Qdrant (HNSW)	5	5	4	5	19
Milvus	5	5	4	4	18
Weaviate	4	4	4	4	16
pgvector (PostgreSQL)	5	3	4	4	16

II.5.4 *Point-in-Time Correctness*

Jaminan *point-in-time correctness* berarti nilai fitur yang dipakai saat pelatihan model harus identik dengan nilai yang tersedia pada saat itu, bukan nilai versi terkini. Wang dan Ruf (2021) memperlihatkan dampak buruk kebocoran temporal pada *backtesting* keuangan, dan analoginya berlaku pada beragam domain. Feast menyediakan operasi `get_historical_features` yang melakukan *point-in-time join* an-

tara entitas dan riwayat fitur, sehingga konsistensi nilai antara fase pelatihan dan fase *inference* dapat ditegakkan.

Tabel II.8 Evaluasi Alternatif *Feature Store Open-Source*

<i>Feature Store</i>	Maturitas	Ekstensibilitas	Komunitas	Integrasi K8s	Total
Feast	5	5	5	5	20
Hopsworks <i>Feature Store</i>	4	4	3	4	15
Feathr (LinkedIn)	3	3	2	3	11
Tecton (basis <i>open-source</i>)	4	3	2	3	12

II.6 Siklus Hidup Model

II.6.1 Eksperimen dan *Tracking*

MLflow (MLflow Contributors 2024) menyediakan *tracking* eksperimen, *registry* model, dan API penyebaran. Pimentel dkk. (2019) menyoroti masalah reproduktibilitas *notebook*, yang menjadi alasan mengapa *tracking* terpusat menjadi prasyarat. Tabel II.9 membandingkan beberapa pilihan platform *tracking*.

Tabel II.9 Evaluasi Alternatif *Experiment Tracking* dan *Model Registry*

<i>Tracking & Registry</i>	Maturitas	Ekstensibilitas	Komunitas	Integrasi	Total
MLflow	5	5	5	5	20
DVC (<i>Data Version Control</i>)	4	4	4	3	15
TensorBoard	4	2	4	2	12
Neptune	3	3	3	3	12

II.6.2 Orkestrasi Pelatihan

Kubeflow Pipelines (Kubeflow Contributors 2024) dan Argo Workflows menjadi dua pilihan utama untuk orkestrasi pelatihan pada Kubernetes. Apache Airflow (Apache Software Foundation 2024a) masih relevan untuk *pipeline* data berorientasi *batch*. Tabel II.10 membandingkan ketiganya berdasarkan kebutuhan domain.

Tabel II.10 Evaluasi Alternatif Orkestrasi ML *Open-Source*

Orkestrasi ML	Maturitas	K8s-Native	Pipeline ML	Komunitas	Total
Kubeflow Pipelines	4	5	5	4	18
Apache Airflow	5	3	3	5	16
Argo Workflows	4	5	3	4	16
Prefect	3	2	3	3	11

II.6.3 Penyajian Model

KServe (KServe Contributors 2024) menyediakan kerangka penyajian model di atas Kubernetes dengan dukungan banyak *runtime*, otomatisasi penskalaan, dan integrasi dengan praktik GitOps. Hsu dkk. (2024) memperlihatkan ujicoba penyebaran terotomasi dengan KServe yang sejalan dengan pendekatan platform ini. Liang dkk. (2024) menyoroti integrasi sistem dengan *machine learning* sebagai pendorong otomatisasi pelatihan dan penyebaran.

Tabel II.11 Evaluasi Alternatif *Model Serving Open-Source*

<i>Model Serving</i>	Maturitas	Skalabilitas	<i>Framework</i>	Komunitas	Total
KServe	4	5	5	4	18
Seldon Core	4	4	4	4	16
BentoML	3	3	4	3	13
TorchServe	3	3	2	3	11

II.7 Tata Kelola Data: Katalog, *Lineage*, dan Kualitas

Bernardo dkk. (2024) memetakan tata kelola data sebagai disiplin lintas fungsi yang mencakup metadata, kualitas, akses, dan kepemilikan. Stoyanovich, Howe, dan Jagdish (2020) memberi konteks etis dengan menempatkan tanggung jawab atas data pada level platform, bukan hanya level aplikasi. Chien (February 2022) memberi panduan praktis untuk peningkatan kualitas data berbasis pengukuran.

DataHub (DataHub Project 2024) dipilih sebagai katalog metadata karena dukungan integrasi luas dengan komponen DataOps dan MLOps. Great Expectations (Great Expectations 2024) dipakai untuk pengujian kontrak data yang dijalankan pada *pipeline* produksi. OpenLineage (OpenLineage Authors 2025) dipakai sebagai standar terbuka pengumpulan *lineage* antar komponen *pipeline*, sehingga jejak dari ingestasi hingga model dapat disusun dengan kosa kata yang seragam. Tabel II.12 merangkum perbandingan beberapa pilihan katalog.

Tabel II.12 Evaluasi Alternatif Manajemen Metadata dan *Governance*

Metadata & Governance	Maturitas	Ekstensibilitas	Komunitas	Integrasi	Total
DataHub (LinkedIn)	5	5	5	5	20
Apache Atlas	4	4	3	4	15
OpenMetadata	3	4	4	4	15
Amundsen (Lyft)	3	3	3	3	12

II.8 Deteksi *Drift*

Lu dkk. (2019) memberikan tinjauan luas tentang pembelajaran di bawah *concept drift*. Bayram, Ahmed, dan Kassler (2022) mengelompokkan keluarga detektor berbasis performa, dan Hinder dkk. (2023) memperluas pembahasan ke arah penjelasan model. Beberapa metrik yang relevan untuk *covariate drift* dan *prior probability shift* antara lain *Population Stability Index* (PSI), uji Kolmogorov-Smirnov (Reis dkk. 2016), dan jarak Wasserstein. Céspedes Sisniega dkk. (2024) memperlihatkan implementasi deteksi *drift* pada lingkungan *serverless*, sementara Jourand dkk. (2023) membahas penanganan *drift* pada *deep learning* untuk pemantauan proses.

Pada platform ini, PSI dan uji Kolmogorov-Smirnov dipakai sebagai detektor utama, dengan ambang batas konfigurasi yang diatur per fitur. Detektor dijalankan oleh *Argo CronWorkflow* sebagai *control loop* yang memicu pelatihan ulang melalui Kubeflow Pipelines apabila ambang dilewati.

II.9 Observabilitas *Cloud-Native*

Cloud Native Computing Foundation (2024b) dan IBM (2024) mendefinisikan observabilitas sebagai gabungan tiga pilar: metrik, log, dan *trace*. Untuk platform ini, Prometheus (Prometheus Authors 2024) dipakai untuk metrik, Loki (Grafana Labs 2024b) untuk log, dan Grafana Tempo (Grafana Labs 2024c) untuk *trace*, dengan OpenTelemetry sebagai *collector* bersama dan Grafana (Grafana Labs 2024a) sebagai antarmuka visualisasi. Sloth (Sloth Authors 2025) membantu menurunkan definisi *service-level objective* ke dalam aturan Prometheus, sedangkan Evidently (Evidently AI 2025) memberi laporan kualitas data dan model yang melengkapi pilar observabilitas. Pyroscope (Grafana Labs 2025) menyediakan *continuous profiling* untuk profil CPU dan memori, Pushgateway (Prometheus Authors 2025) menjadi titik kumpul metrik untuk *job* berdurasi pendek di luar siklus *scrape*, dan OpenCost (OpenCost Authors 2025) memantau biaya per beban kerja pada kluster. Apache Superset (Apache Superset Authors 2025) berperan sebagai antarmuka BI yang menyatukan kueri SQL lintas ClickHouse, Trino, dan MySQL pada satu la-

pis dasbor; Tabel II.13 merangkum perbandingan beberapa pilihan *BI open-source*. Pemantauan tambahan untuk kualitas data dan model dirangkai sebagai *dashboard* pada Grafana sehingga pengamatan terhadap kondisi sistem dapat dilakukan dari satu antarmuka. Tabel II.14 merangkum perbandingan beberapa pilihan tumpukan observabilitas.

Tabel II.13 Evaluasi Alternatif *Business Intelligence* dan *Dashboarding Open-Source*

<i>BI Tool</i>	Maturitas	Konektor SQL	Multi-Tenant K8s	Komunitas	Total
Apache Superset	5	5	5	5	20
Metabase	4	4	4	5	17
Redash	4	4	3	3	14
Lightdash	3	4	3	3	13

Tabel II.14 Evaluasi Alternatif *Monitoring* dan *Observability*

<i>Monitoring & Observability</i>	Maturitas	Ekstensibilitas	Komunitas	Integrasi K8s	Total
Prometheus + Grafana	5	5	5	5	20
InfluxDB + Chronograf	4	4	3	4	15
Elastic Stack (ELK)	5	4	4	4	17
Jaeger + OpenTelemetry	4	4	4	5	17

II.10 GitOps dan *Continuous Delivery*

GitOps menempatkan repositori Git sebagai sumber kebenaran tunggal untuk konfigurasi sistem. Limoncelli (2022) menjelaskan pola GitOps sebagai pendekatan IT *self-service* yang memetakan operasi ke kontrol versi. Argo CD (Argo Project 2024) dipakai sebagai *operator* GitOps yang merekonsiliasi keadaan *cluster* dengan deklarasi pada Git. Kim dkk. (2016) memberi dasar teoretis tentang aliran kerja CI-/CD yang menjadi induk GitOps. Tekton (Tekton Contributors 2024) dipakai sebagai *pipeline* CI yang membangun *image* dan menjalankan pengujian sebelum komit menjadi sumber penyebaran. Argo Rollouts (Argo Rollouts Authors 2025) melengkapi rantai dengan strategi penyebaran progresif berbasis *canary* dan analisis metrik. Tabel II.15 merangkum perbandingan beberapa *GitOps controller open-source* berdasarkan kriteria yang relevan dengan platform.

Tabel II.15 Evaluasi Alternatif *GitOps Controller Open-Source*

<i>GitOps Controller</i>	Maturitas	K8s-Native	Antarmuka	Komunitas	Total
Argo CD	5	5	5	5	20
Flux CD	5	5	3	4	17
Rancher Fleet	4	5	3	3	15
Jenkins X	3	4	3	2	12

II.11 Keamanan Platform: Identitas, Rahasia, *Mesh*, dan Kebijakan

Lapisan keamanan menjadi prasyarat bagi platform yang melayani DataOps dan MLOps secara berdampingan. Empat aspek dipertimbangkan: identitas pengguna terpadu, pengelolaan rahasia, *service mesh*, dan *policy engine*.

Untuk identitas, Dex (Dex Authors 2025) dipakai sebagai *identity provider* berbasis OpenID Connect (OIDC) karena *footprint* ringan, dukungan federasi terhadap penyedia identitas eksternal, dan integrasi sederhana dengan *cluster* Kubernetes. Tabel II.16 merangkum perbandingan beberapa *identity provider open-source* berdasarkan kriteria yang relevan dengan platform.

Tabel II.16 Evaluasi Alternatif *Identity Provider Open-Source*

<i>Identity Provider</i>	Bobot <i>Footprint</i>	Federasi OIDC	Operabilitas K8s	Maturitas	Total
Dex <i>IdP</i>	5	5	5	5	20
Keycloak	3	5	4	5	17
Authelia	4	4	3	4	15
Authentik	3	4	3	4	14

Untuk pengelolaan rahasia, OpenBao (OpenBao Authors 2025) dipakai sebagai *successor open-source* dari HashiCorp Vault yang menjamin penyimpanan rahasia terenkripsi serta penerbitan kredensial dinamis. Integrasi pada beban kerja dilakukan melalui *External Secrets Operator* yang menyalin rahasia ke Secret Kubernetes dengan rotasi otomatis. Tabel II.17 merangkum perbandingan beberapa pengelola rahasia *open-source*.

Tabel II.17 Evaluasi Alternatif Pengelola Rahasia *Open-Source*

Pengelola Rahasia	Lisensi <i>Open</i>	Rotasi Dinamis	Integrasi ESO	Maturitas	Total
OpenBao	5	5	5	4	19
HashiCorp Vault <i>Community</i>	3	5	5	5	18
Bitnami <i>Sealed Secrets</i>	5	2	3	5	15
Mozilla SOPS	5	2	3	4	14

Untuk komunikasi antar-layanan, Istio (Istio Authors 2024) dipakai sebagai *service mesh* yang menyediakan *mutual TLS* (mTLS) seragam, kontrol L7 berbasis VirtualService dan AuthorizationPolicy, serta integrasi telemetri dengan OpenTelemetry. Tabel II.18 merangkum perbandingan beberapa *service mesh open-source* berdasarkan kriteria fitur, ekosistem, dan kematangan. Pada perbatasan *mesh*, Apache APISIX (Apache APISIX Authors 2025) berperan sebagai *API gateway* yang menyaring lalu lintas masuk, menerapkan kebijakan kuota, dan menjembatani mTLS terhadap konsumen di luar *mesh*. Tabel II.19 merangkum perbandingan beberapa *API gateway open-source*.

Tabel II.18 Evaluasi Alternatif *Service Mesh Open-Source*

<i>Service Mesh</i>	Maturitas	Fitur L7	Ekosistem CRD	Komunitas	Total
Istio <i>ambient/sidecar</i>	5	5	5	5	20
Linkerd	4	3	3	4	14
Cilium Service Mesh	4	4	4	4	16
Consul Connect	4	3	3	3	13

Tabel II.19 Evaluasi Alternatif *API Gateway Open-Source*

<i>API Gateway</i>	Maturitas	Lisensi <i>Open</i>	Operabilitas K8s	Plugin/Ekstensi	Total
Apache APISIX	5	5	5	5	20
Kong <i>Gateway OSS</i>	5	4	4	5	18
Emissary Ingress (Ambassador)	4	5	4	3	16
NGINX Ingress	5	5	4	3	17

Untuk penegakan kebijakan, Kyverno (Kyverno Authors 2024) dipakai sebagai *policy engine native* Kubernetes dengan sintaks YAML yang ringkas serta dukungan

mutate, *validate*, dan *generate*. Open Policy Agent (OPA) Gatekeeper (Open Policy Agent Authors 2024) juga umum dipakai sebagai alternatif berbasis Rego. Tabel II.20 merangkum perbandingan beberapa *policy engine open-source*.

Tabel II.20 Evaluasi Alternatif *Policy Engine Open-Source*

<i>Policy Engine</i>	Sintaks Native	Mode Mutate	Pelaporan Pelanggaran	Komunitas	Total
Kyverno	5	5	5	5	20
OPA Gatekeeper	3	4	4	5	16
jsPolicy	4	3	3	3	13
Kubewarden	4	4	3	3	14

Untuk pemantauan ancaman pada lapis *runtime*, Falco (Falco Authors 2025) dipakai sebagai detektor berbasis eBPF yang memantau *syscall* dan menerbitkan peristiwa keamanan, sedangkan Trivy Operator (Aqua Security 2025) memindai *image* dan beban kerja secara terjadwal. Pencadangan *cluster* dan *persistent volume* dijaga oleh Velero (Velero Authors 2025), dan uji ketahanan dijalankan dengan Chaos Mesh (Chaos Mesh Authors 2025) pada *namespace* eksperimen. Tabel II.21 merangkum perbandingan beberapa *runtime security open-source*.

Tabel II.21 Evaluasi Alternatif *Runtime Security Open-Source* pada Kubernetes

<i>Runtime Security</i>	Maturitas	Cakupan eBPF/Kernel	Aturan Bawaan	Komunitas	Total
Falco (CNCF)	5	5	5	5	20
Cilium Tetragon	4	5	4	4	17
Aqua Tracee	3	5	3	3	14
Sysdig Secure OSS	4	4	4	3	15

II.12 Penelitian Terkait dan Posisi Penelitian

Beberapa penelitian terdahulu menjadi tetangga dekat platform ini. Burgueno-Romero, Cánovas Izquierdo, dan García-García (2025) mengusulkan arsitektur MLOps *open source*, namun belum memasukkan layanan fitur dengan dukungan vektor sebagai komponen kelas pertama. Amou Najafabadi dkk. (2024) memetakan beragam arsitektur MLOps tetapi tidak menggabungkan DataOps secara menyeluruh. Rodrigues dkk. (2025) menyentuh *near real-time stream learning*, namun tidak meletakkan tata kelola data sebagai bidang kendali yang ditegakkan. Ahmad dkk. (June 2024) membahas strategi keamanan MLOps, dan dengan demikian melengkapi sudut pandang yang dibahas pada platform ini di bagian kebijakan jaringan dan manajemen rahasia.

Platform yang dikembangkan pada Tugas Akhir ini menggabungkan tiga karakter yang belum dilingkupi sekaligus pada penelitian sebelumnya: integrasi DataOps dan MLOps pada satu bidang kendali Kubernetes, layanan fitur *dual-store* dengan dukungan vektor HNSW, dan deteksi *drift* terotomasi yang memicu pelatihan ulang melalui kontrol GitOps. Tabel II.22 menyajikan posisi platform ini terhadap penelitian terkait.

Tabel II.22 Perbandingan Detail Arsitektur Saat Ini dengan yang Diusulkan

Aspek	Arsitektur Saat Ini	Arsitektur Diusulkan	Keuntungan Terukur	Referensi
Orkestrasi	<i>Compute</i> heterogen dengan <i>provisioning</i> manual, waktu tunggu <i>scaling</i> panjang, dan tingkat standardisasi rendah.	Platform Kubernetes terpadu dengan manajemen deklaratif dan alokasi sumber daya otomatis, sejalan dengan komponen arsitektur MLOps yang diidentifikasi Kreuzberger, Kühl, dan Hirschl (2023). GitOps dengan Argo CD memastikan perubahan infrastruktur dan aplikasi mengikuti <i>single source of truth</i> di Git.	Lakka (2025) menunjukkan bahwa adopsi Kubernetes pada perusahaan disertai praktik DevOps yang matang meningkatkan <i>deployment frequency</i> dan menekan <i>Mean Time To Recovery</i> (MTTR) secara signifikan; arsitektur yang diusulkan menargetkan karakteristik performa pada arah yang sama.	(Kreuzberger, Kühl, dan Hirschl 2023; Lakka 2025)
Manajemen Fitur	Logika fitur tersebar di <i>notebook</i> dan skrip dengan duplikasi tinggi dan banyak fungsi yang saling tumpang tindih.	Feast dengan <i>feature registry</i> terpusat, <i>dual serving offline/online</i> , serta dukungan <i>point-in-time correctness</i> .	Duplikasi fitur dapat ditekan, <i>training-serving skew</i> berkurang melalui pemisahan <i>offline/online</i> dan <i>point-in-time retrieval</i> , dan <i>feature reuse</i> berpotensi meningkat signifikan.	(Munappy dkk. 2022; Polyzotis dkk. 2018)

Aspek	Arsitektur Saat Ini	Arsitektur Diusulkan	Keuntungan Terukur	Referensi
<i>Vector Embeddings</i>	Fitur <i>embedding</i> tidak didukung secara <i>native</i> atau diimplementasikan secara kustom dengan latensi tinggi dan <i>throughput</i> terbatas.	Qdrant sebagai indeks vektor terpisah dari <i>online store</i> tabular Valkey, dengan HNSW <i>native</i> , <i>API gRPC</i> , dan filter <i>payload</i> .	Johnson, Douze, dan Jégou (2021) menunjukkan bahwa indeks vektor berbasis HNSW maupun FAISS dapat melayani <i>billion-scale similarity search</i> dengan latensi rendah; desain ini memakai Qdrant agar fitur <i>embedding</i> dilayani secara <i>online</i> tanpa membebani <i>online store</i> kunci-nilai dan tanpa membangun layanan kustom.	(Johnson, Douze, dan Jégou 2021; Qdrant Team 2025)
Validitas Temporal	<i>Point-in-time join</i> dilakukan manual dan rentan kesalahan, sehingga banyak tim mengalami <i>data leakage</i> .	<i>Point-in-time join</i> otomatis dari Feast dengan pola verifikasi yang lebih sistematis.	Pendekatan ini mencegah kebocoran data temporal yang dapat menghasilkan estimasi performa <i>backtest</i> yang terlalu optimistis dan tidak realistis di produksi.	(Polyzotis dkk. 2018)

Aspek	Arsitektur Saat Ini	Arsitektur Diusulkan	Keuntungan Terukur	Referensi
Pemrosesan Data	<i>Batch</i> dan <i>stream</i> diproses oleh <i>pipeline</i> terpisah tanpa arsitektur yang koheren, sehingga hasil pemrosesan tidak konsisten.	Spark sebagai mesin <i>batch processing</i> dan Flink sebagai mesin <i>stream processing</i> yang hasilnya dimaterialisasikan langsung ke <i>Feature Store</i> . Tekton dan Argo Workflows mengorkestrasi <i>pipeline</i> berbasis manifes deklaratif sehingga pengembangan dan penjadwalan transformasi menjadi lebih sistematis.	Waktu pengembangan <i>pipeline</i> dapat menurun dan <i>freshness</i> data meningkat berkat pemrosesan <i>streaming</i> yang lebih efisien dan terintegrasi.	(Rodrigues dkk. 2025)
Pelatihan Model	<i>Workflow</i> pelatihan berbasis <i>notebook</i> dengan <i>tracking</i> eksperimen informal dan <i>artifact</i> tidak terkelola.	Kubeflow <i>Pipelines</i> untuk orkestrasi pelatihan dan MLflow untuk <i>tracking</i> eksperimen.	Reprodusibilitas meningkat melalui kontainerisasi dan pengelolaan eksperimen terpusat, sementara siklus pengembangan <i>pipeline</i> menjadi lebih terstruktur.	(Amershi dkk. 2019; Pimentel dkk. 2019)

Aspek	Arsitektur Saat Ini	Arsitektur Diusulkan	Keuntungan Terukur	Referensi
Deployment Model	Proses <i>deployment</i> manual dengan <i>refactoring</i> besar dan sering terjadi <i>bug</i> pasca- <i>deploy</i> .	KServe dengan <i>InferenceService</i> deklaratif dan <i>deployment</i> otomatis melalui GitOps di atas Knative Serving. OpenTelemetry mengumpulkan log dan <i>trace</i> inferensi ke Loki dan Grafana Tempo sebagai dasar <i>monitoring</i> dan analitik lanjutan.	Waktu <i>deployment</i> berpotensi turun secara signifikan, dan insiden terkait kesalahan konfigurasi <i>deployment</i> dapat dikurangi lewat otomatisasi.	(Paleyes, Urma, dan Lawrence 2022)
Pola Deployment Lanjut	Pola seperti <i>canary release</i> jarang digunakan, organisasi umumnya mengandalkan <i>binary switchover</i> .	Dukungan <i>canary release</i> , A/B <i>testing</i> , dan <i>automatic rollback</i> prediktif.	<i>Rollout</i> dapat dilakukan secara bertahap dan aman, sehingga MTTR menurun karena kegagalan dapat dibatasi pada <i>canary traffic</i> .	(Shankar dkk. 2022)
Deteksi Drift	Deteksi <i>drift</i> bersifat reaktif dengan jeda deteksi yang panjang terhadap isu penting dan perubahan gradual.	<i>Monitoring</i> proaktif dengan uji statistik, <i>alert real-time</i> , dan penetapan ambang adaptif.	Jeda deteksi <i>drift</i> dapat dipersingkat sehingga <i>retraining</i> dapat dijadwalkan lebih tepat waktu untuk menjaga kinerja model.	(Céspedes Sisniega dkk. 2024; Bayram, Ahmed, dan Kassler 2022)
Governance & Lineage	Dokumentasi dilakukan secara manual dan <i>audit trail</i> sering tidak lengkap atau tersebar.	DataHub dengan <i>metadata ingestion</i> otomatis dan <i>lineage</i> yang dapat ditelusuri dari <i>source</i> hingga model, dibantu OpenLineage sebagai protokol pengirim peristiwa lintas <i>engine</i> pemrosesan.	Waktu persiapan audit dapat berkurang dan cakupan metadata meningkat melalui proses <i>ingestion</i> dan <i>lineage</i> otomatis.	(Stoyanovich, Howe, dan Jagadish 2020; Lamer dkk. 2024)

Aspek	Arsitektur Saat Ini	Arsitektur Diusulkan	Keuntungan Terukur	Referensi
<i>Observability</i>	Fokus <i>monitoring</i> pada metrik infrastruktur, tanpa cakupan metrik ML spesifik di sebagian besar organisasi.	<i>Observability</i> mencakup metrik ML, <i>dashboard</i> kinerja model, dan <i>alert</i> berbasis indikator <i>drift</i> . Prometheus dan Grafana menjadi inti, OpenTelemetry mengalirkan log dan <i>trace</i> ke Loki dan Grafana Tempo, sementara Alertmanager mengonsolidasikan <i>alert</i> ke berbagai kanal notifikasi.	Proses <i>debugging</i> dan <i>root cause analysis</i> menjadi lebih efektif sehingga MTTR dapat diturunkan.	(Shankar dkk. 2022)
Integrasi CI/CD	CI/CD untuk <i>pipeline</i> dan model dilakukan secara manual atau semi-otomatis, dengan <i>workflow</i> yang tidak seragam.	GitOps dengan Argo CD sebagai mekanisme <i>continuous deployment</i> deklaratif, dilengkapi Tekton untuk <i>continuous integration</i> dan Argo Rollouts untuk <i>progressive delivery</i> , dengan Git sebagai sumber kebenaran tunggal yang mencakup seluruh konfigurasi DataOps dan MLOps.	Frekuensi <i>deployment</i> dapat meningkat dan variasi kesalahan <i>deployment</i> dapat ditekan melalui otomasi dan <i>peer review</i> pada <i>pull request</i> .	(Kreuzberger, Kühl, dan Hirschl 2023)

Aspek	Arsitektur Saat Ini	Arsitektur Diusulkan	Keuntungan Terukur	Referensi
Pengalaman <i>Developer</i>	<i>Developer</i> menghabiskan banyak waktu untuk tugas repetitif, <i>onboarding</i> lama, dan penggunaan alat yang terpisah.	Platform terintegrasi dengan <i>self-service</i> untuk data, fitur, <i>pipeline</i> , dan akses; dex sebagai <i>identity provider</i> OIDC dan oauth2-proxy menyatukan otentikasi tunggal di seluruh antarmuka pengembang.	<i>Onboarding</i> dapat dipercepat dan produktivitas <i>data scientist</i> meningkat karena hambatan akses infrastruktur berkurang.	(Paleyes, Urma, dan Lawrence 2022; Rella 2022)
Efisiensi Biaya	Utilisasi sumber daya tidak optimal dan tidak ada atribusi biaya yang jelas ke <i>workload</i> .	<i>Auto-scaling</i> berbasis HPA, VPA, dan KEDA, pemantauan biaya melalui OpenCost, serta alokasi sumber daya yang lebih terukur pada kluster Kubernetes. Seluruh komponen inti berbasis <i>open source</i> sehingga biaya lisensi dapat ditekan.	Penghematan biaya signifikan dapat dicapai melalui optimasi pemakaian sumber daya dan <i>auto-scaling</i> , sebagaimana ditunjukkan dalam berbagai studi kasus adopsi Kubernetes (Lakka 2025); desain ini mengadopsi prinsip yang sama untuk mengoptimalkan pemakaian sumber daya.	(Lakka 2025)

Aspek	Arsitektur Saat Ini	Arsitektur Diusulkan	Keuntungan Terukur	Referensi
Keamanan & Compliance	Kontrol akses beragam di tiap sistem, dan <i>audit</i> tersebar tanpa pandangan terpadu.	RBAC terintegrasi, <i>audit trail</i> komprehensif, dan pengecekan <i>compliance</i> yang terotomatisasi. dex sebagai <i>identity provider</i> OIDC memusatkan identitas, oauth2-proxy menegakkan otentikasi pada antarmuka aplikasi, Kyverno menegakkan kebijakan admisi pada klaster, OpenBao mengelola <i>secrets</i> , sementara Falco dan Trivy Operator memantau ancaman <i>runtime</i> dan kerentanan citra.	Risiko insiden keamanan dapat berkurang melalui kontrol terpusat dan proses audit menjadi lebih ringkas.	(Ahmad dkk. June 2024)

BAB III

ANALISIS MASALAH

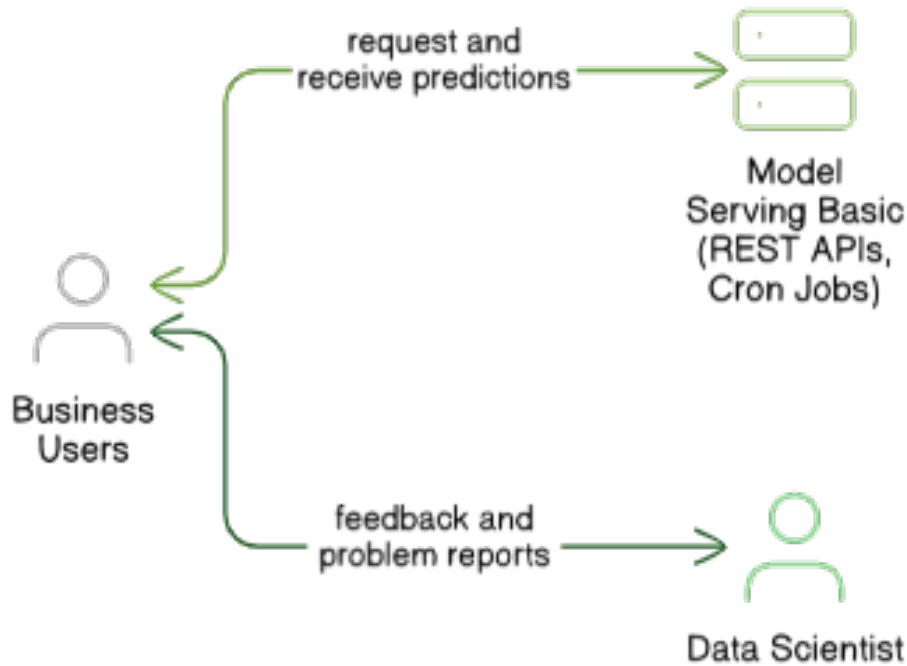
Bab ini membahas analisis masalah pada kondisi pengembangan model *machine learning* dengan dukungan data berskala produksi, kebutuhan fungsional dan non-fungsional bagi platform DataOps dan MLOps terintegrasi, serta proses pemilihan solusi yang paling sesuai dengan tujuan penelitian. Pembahasan disusun secara berurutan, mulai dari pemetaan kondisi saat ini, identifikasi masalah pengguna, formulasi kebutuhan, hingga evaluasi alternatif solusi pada *control plane* Kubernetes.

III.1 Analisis Kondisi Saat Ini

Pengembangan model *machine learning* untuk tahap produksi melibatkan empat peran utama, yaitu *business user*, *data engineer*, *data scientist*, dan *machine learning engineer*. Kreuzberger, Kühl, dan Hirschl (2023) memperlihatkan bahwa peran tersebut sering bekerja pada *tool* yang berbeda, sehingga muncul fragmentasi yang menghambat kolaborasi lintas tim. Paleyes, Urma, dan Lawrence (2022) menambahkan bahwa fragmentasi ini melahirkan integrasi yang lemah pada sambungan data dan model, dan pada akhirnya menurunkan kecepatan rilis serta kualitas keluaran.

III.1.1 Fragmentasi pada Peran *Business User*

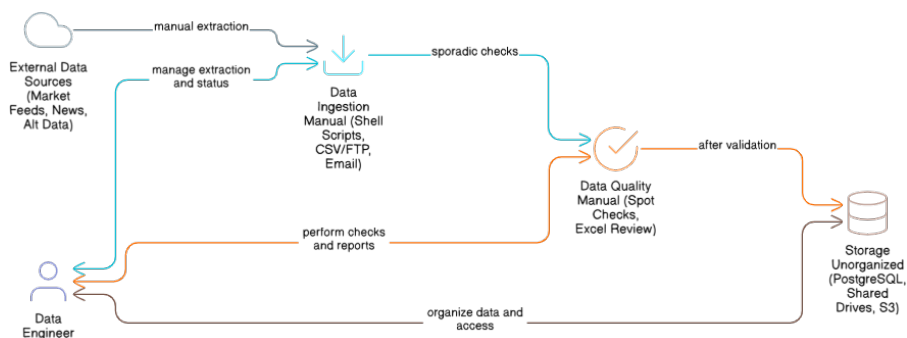
Peran *business user* biasanya berinteraksi dengan tumpukan visualisasi dan pelaporan yang terpisah dari sumber data analitik dan model. Gambar III.1 memperlihatkan posisi peran ini pada arsitektur fragmentasi dengan akses utama melalui *dashboard* dan laporan. Konsekuensinya, informasi tentang metrik *drift*, kualitas data, dan kebijakan akses sering tidak tersaji dalam satu antarmuka, sehingga keputusan bisnis mengandalkan ringkasan berkala yang tidak selalu terhubung dengan kondisi sistem terkini.



Gambar III.1 Fragmentasi pada Peran *Business User*

III.1.2 Fragmentasi pada Peran *Data Engineer*

Data engineer bertanggung jawab atas ingestasi, pemrosesan, dan penyimpanan data. Gambar III.2 memperlihatkan luasnya cakupan tugas pada lapisan ingestasi, transformasi, dan tata kelola. Lamer dkk. (2024) menyoroti kompleksitas tata kelola data yang sering muncul ketika *tool* ingestasi, kualitas, dan katalog tidak terintegrasi pada satu sumber kebenaran. Akibatnya, kebijakan kualitas dan *lineage* antar lapisan data harus disinkronkan secara manual.

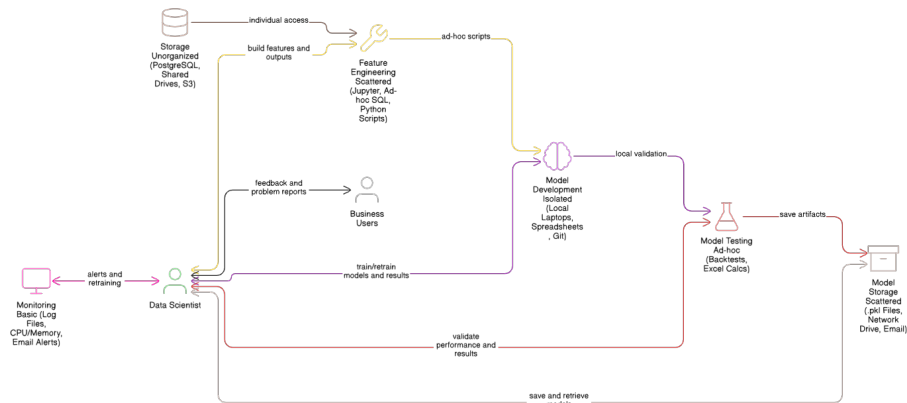


Gambar III.2 Fragmentasi pada Peran *Data Engineer*

III.1.3 Fragmentasi pada Peran *Data Scientist*

Peran *data scientist* menghabiskan sebagian besar waktu pada eksperimen dan rekayasa fitur. Gambar III.3 memperlihatkan ketergantungan pada *notebook*, *experiment*

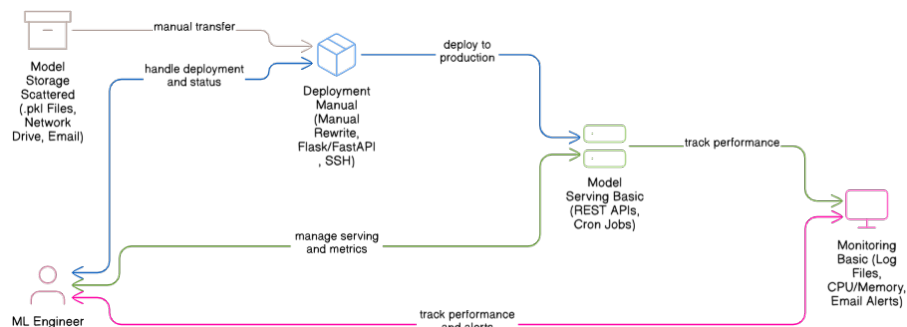
tracking, dan *feature engineering* yang sering berdiri sendiri. Anaconda, Inc. (July 2020) dan Pimentel dkk. (2019) memperlihatkan bahwa reproduksibilitas eksperimen terganggu apabila lingkungan eksekusi, versi data, dan versi kode tidak diatur dalam satu *pipeline* yang sama, dan kondisi ini diperburuk oleh *technical debt* yang khas pada sistem *machine learning* sebagaimana dipetakan oleh Sculley dkk. (2015).



Gambar III.3 Fragmentasi pada Peran *Data Scientist*

III.1.4 Fragmentasi pada Peran *Machine Learning Engineer*

Machine learning engineer bertugas membawa model dari fase eksperimen ke fase produksi. Gambar III.4 memperlihatkan kebutuhan pada *model registry*, *serving*, dan pemantauan model. Rella (2022) memperlihatkan dampak buruk yang muncul ketika siklus hidup model tidak terhubung dengan kontrol versi data dan fitur. Akibatnya, mekanisme *rollback*, *canary*, dan *retraining* tidak konsisten antar tim.

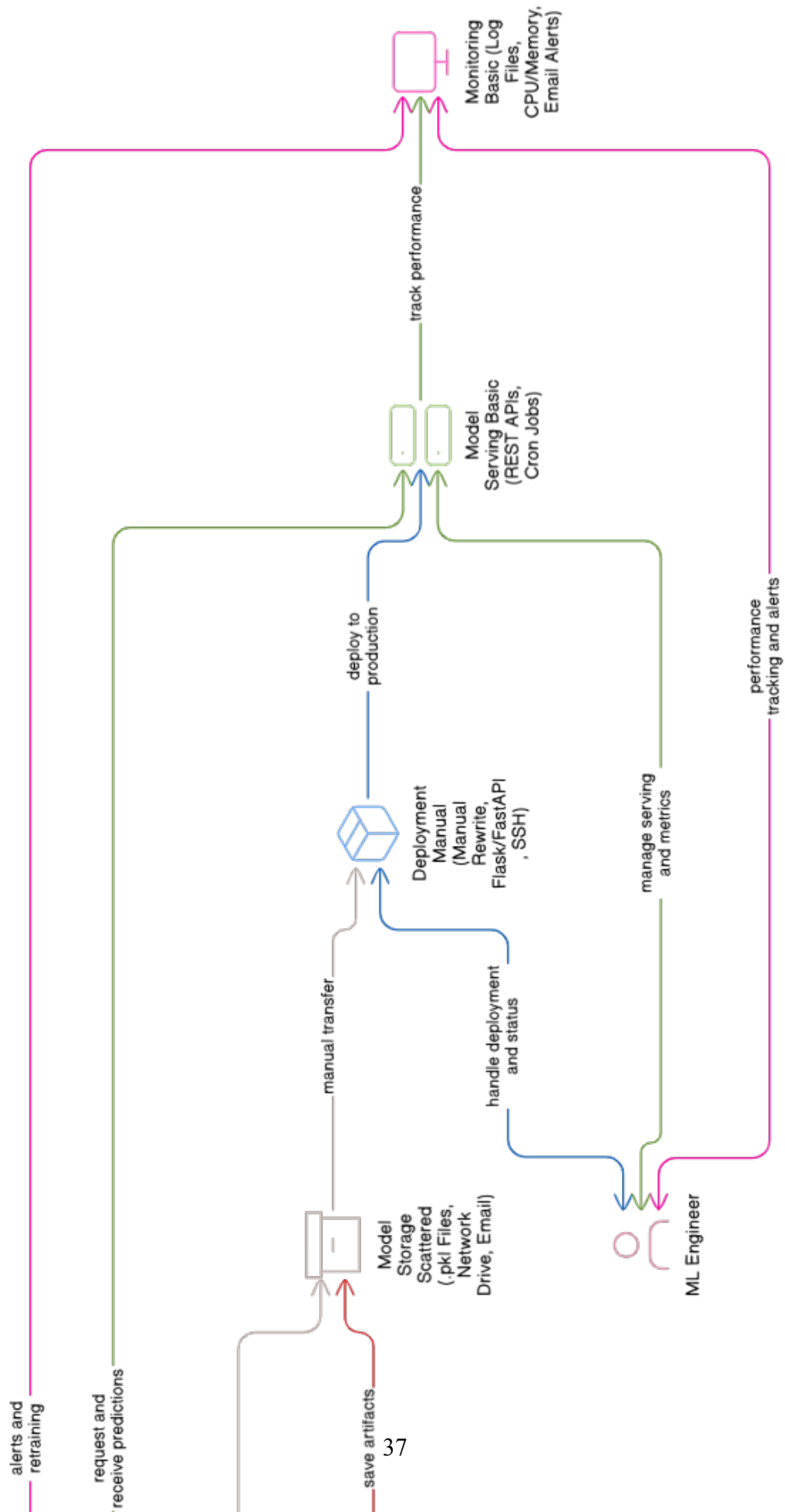


Gambar III.4 Fragmentasi pada Peran *Machine Learning Engineer*

III.1.5 Pandangan Umum Kondisi Fragmentasi

Gambar III.5 menggabungkan empat peran tersebut dalam satu pandangan menyeluruh. Amou Najafabadi dkk. (2024) dan Burgueno-Romero, Cánovas Izquierdo,

dan García-García (2025) memperlihatkan bahwa fragmentasi tidak hanya muncul pada antarmuka tim, tetapi juga pada bidang kendali infrastruktur, kebijakan akses, dan observabilitas. Kondisi inilah yang menjadi titik tolak penelitian, karena empat masalah inti yang dirumuskan pada Bab I pada hakikatnya merupakan konsekuensi dari fragmentasi tersebut, yaitu fragmentasi *tool*, ketidakkonsistenan tata kelola, kombinasi *drift* dan *temporal leakage*, serta keterbatasan dalam menyajikan fitur multimoda dari satu sumber kebenaran.



III.2 Analisis Kebutuhan

Setelah pemetaan kondisi saat ini, kebutuhan platform dirumuskan dalam dua kategori: kebutuhan fungsional yang menggambarkan fitur yang harus dimiliki platform, dan kebutuhan nonfungsional yang menggambarkan kualitas atributif yang harus dipenuhi. Identifikasi kebutuhan menggunakan empat rumusan masalah dari Bab I sebagai titik berangkat.

III.2.1 Identifikasi Masalah Pengguna

Identifikasi masalah pengguna dijabarkan dengan menelusuri empat rumusan masalah. Pertama, pengguna pada lintas peran membutuhkan satu bidang kendali untuk mengatur ingestasi, pelatihan, penyajian, dan pemantauan, sehingga fragmentasi *tool* dapat dikurangi. Kedua, pengguna pada peran *data engineer* dan *data steward* membutuhkan tata kelola yang seragam di antara lapisan data, fitur, dan model, mencakup *lineage*, katalog, dan validasi kualitas. Ketiga, pengguna pada peran *data scientist* dan *machine learning engineer* membutuhkan deteksi *drift* yang terotomasi sekaligus jaminan *point-in-time correctness* agar evaluasi model tidak mengandung kebocoran temporal sebagaimana ditegaskan oleh Vela dkk. (2022) dan Wang dan Ruf (2021). Keempat, pengguna pada peran *data scientist* membutuhkan dukungan fitur multimoda yang terdiri dari fitur tabular, urutan waktu, dan *embedding* berdimensi tinggi dengan latensi rendah pada saat *inference*, sebagaimana diperlihatkan oleh Devlin dkk. (2019), OpenAI (December 2022), dan Malkov dan Yashunin (2020).

III.2.2 Kebutuhan Fungsional

Tabel III.1 menyajikan empat belas kebutuhan fungsional. Manajemen fitur terpusat, penyajian fitur ganda *online-offline*, jaminan validitas temporal, dan dukungan *vector embedding* merangkum kebutuhan pada lapisan fitur. Otomatisasi *pipeline* pelatihan, otomatisasi penerapan model, pelacakan eksperimen, dan registri model merangkum kebutuhan pada siklus hidup model. Pemantauan *drift* otomatis, *governance* dan penemuan, pemrosesan *batch* dan *stream*, konfigurasi deklaratif, API dan SDK terpadu, serta manajemen sumber daya merangkum kebutuhan pada lapisan operasional. Daftar ini dipetakan langsung pada empat rumusan masalah dan empat tujuan dari Bab I.

Tabel III.1 Kebutuhan Fungsional Sistem

ID	Kebutuhan Fungsional	Deskripsi	Prioritas
KF-01	Manajemen Fitur Terpusat	Sistem menyediakan <i>feature registry</i> terpusat untuk mendefinisikan, mengelola, dan melakukan <i>versioning</i> fitur. <i>Registry</i> menyimpan metadata seperti deskripsi, tipe data, <i>owner</i> , dan dependensi, serta dikelola di dalam repositori Git agar mendukung <i>workflow</i> GitOps dan penelusuran perubahan.	Tinggi
KF-02	Penyajian Fitur Ganda	Sistem mampu melakukan <i>serving</i> fitur dalam dua mode, yaitu <i>online serving</i> untuk <i>real-time inference</i> dengan target latensi < 10 ms di p99 dan <i>offline serving</i> untuk <i>historical retrieval</i> berkapasitas besar yang mendukung <i>point-in-time correct join</i> pada data historis.	Tinggi
KF-03	Jaminan Validitas Temporal	Sistem secara bawaan menjamin <i>point-in-time correctness</i> ketika menyusun <i>training dataset</i> untuk data <i>time-series</i> , sehingga mencegah kebocoran data temporal yang dapat membuat evaluasi dan model menjadi tidak valid.	Tinggi
KF-04	Dukungan <i>Vector Embeddings</i>	Sistem mendukung penyimpanan, pengindeksan, dan <i>serving vector embeddings</i> pada <i>online store</i> dengan <i>similarity search</i> yang efisien (latensi < 10 ms di p99), misalnya melalui algoritme HNSW atau metode sejenis.	Tinggi
KF-05	Otomatisasi <i>Pipeline</i> Pelatihan	Sistem menyediakan <i>pipeline</i> otomatis untuk mengorkestrasi proses pelatihan model dari ujung ke ujung, terintegrasi dengan <i>feature store</i> , <i>experiment tracking</i> , dan <i>model registry</i> , sehingga mendukung pencapaian otomasi sesuai MLOps Level 2.	Tinggi
KF-06	Otomatisasi Penerapan Model	Sistem mengotomatisasi <i>deployment</i> model yang telah lulus validasi ke <i>inference endpoint</i> produksi, serta mendukung strategi <i>rolling update</i> , <i>canary deployment</i> , <i>A/B testing</i> , dan mekanisme <i>rollback</i> yang aman ketika terjadi degradasi.	Tinggi
KF-07	Pemantauan <i>Drift</i> Otomatis	Sistem secara otomatis dan berkala memantau <i>data drift</i> dan <i>concept drift</i> dengan membandingkan distribusi fitur di produksi terhadap distribusi pada saat pelatihan menggunakan uji statistik (misalnya PSI, K-S) dengan ambang adaptif, serta terintegrasi dengan <i>event-driven monitoring</i> untuk memungkinkan respon otomatis terhadap anomali.	Tinggi
KF-08	<i>Governance</i> & Penemuan Otomatis	Sistem menangkap dan menyajikan metadata serta <i>lineage</i> untuk fitur, <i>dataset</i> , model, dan <i>pipeline</i> dalam sebuah katalog yang dapat dicari, sehingga memudahkan <i>discovery</i> , analisis dampak, dan pemenuhan kebutuhan <i>compliance</i> .	Tinggi

ID	Kebutuhan Fungsional	Deskripsi	Prioritas
KF-09	Pelacakan Eksperimen Terintegrasi	Sistem menyediakan <i>experiment tracking</i> yang terintegrasi untuk mencatat parameter, metrik, artefak, dan versi kode, sehingga proses eksperimen dapat direproduksi dan dibandingkan secara sistematis.	Tinggi
KF-10	<i>Versioning</i> dan Registri Model	Sistem menyediakan <i>model registry</i> terpusat untuk mengelola versi model dan status siklus hidupnya (<i>staging, production, archived</i>) disertai <i>audit trail</i> lengkap untuk mendukung <i>governance</i> .	Tinggi
KF-11	Pemrosesan <i>Batch</i> dan <i>Stream</i>	Sistem mendukung <i>batch processing</i> untuk analisis dan rekayasa fitur historis serta <i>stream processing</i> untuk komputasi fitur <i>real-time</i> , di mana keduanya terhubung dengan <i>feature store</i> sehingga definisi fitur tetap konsisten.	Tinggi
KF-12	Konfigurasi Deklaratif	Sistem memungkinkan konfigurasi komponen seperti fitur, <i>pipeline</i> , dan <i>deployment</i> dalam format deklaratif (misalnya YAML atau Python), yang dikontrol versinya di Git dan selaras dengan prinsip GitOps.	Tinggi
KF-13	API dan SDK Terpadu	Sistem menyediakan API dan SDK terpadu (misalnya untuk Python) sebagai antarmuka standar untuk berinteraksi dengan seluruh komponen, sehingga kompleksitas integrasi menurun dan pola pemakaian menjadi konsisten.	Sedang
KF-14	Manajemen Sumber Daya	Sistem menyediakan kemampuan pengelolaan sumber daya komputasi yang efisien, termasuk <i>resource quota</i> , prioritas, serta <i>dynamic provisioning</i> pada lingkungan <i>multi-tenant</i> untuk mencegah satu <i>workload</i> menghabiskan sumber daya klaster.	Sedang

III.2.3 Kebutuhan Nonfungsional

Tabel III.2 menyajikan dua belas kebutuhan nonfungsional yang berperan sebagai pembatas kualitas. Kinerja latensi rendah dan *throughput* tinggi menyangkut performa penyajian fitur dan *stream processing*. Skalabilitas horizontal, keandalan, dan toleransi kesalahan menyangkut kemampuan platform mengikuti pertumbuhan beban. Konsistensi dan kebenaran menjamin tidak ada kebocoran temporal. *Observability* mengikat metrik, log, dan *trace* pada satu antarmuka. Keamanan mengikat autentikasi, otorisasi, enkripsi, dan *audit logging*. Ekstensibilitas dan modularitas memberi ruang bagi penggantian komponen tanpa mengganggu komponen lain. Kemudahan penggunaan, efisiensi sumber daya, portabilitas, dan kemudahan pemeliharaan menutup daftar dengan menyoroti aspek operasional jangka panjang. Target metrik untuk lapisan inferensi mengikuti panduan Bondi (2000) terkait karakteristik skalabilitas dan pengaruhnya pada kinerja.

Tabel III.2 Kebutuhan Nonfungsional Sistem

ID	Kebutuhan Nonfungsional	Deskripsi	Target Metrik
KNF-01	Kinerja Latensi Rendah	<i>Online feature serving</i> dan <i>vector similarity search</i> harus memberikan latensi yang stabil dan rendah untuk mendukung inferensi waktu nyata pada berbagai domain aplikasi. Layanan ini perlu mampu merespons permintaan dengan cepat, bahkan pada saat beban tinggi.	Latensi p99 sub-detik pada <i>feature serving</i> dan <i>vector search</i> ; ambang spesifik per layanan diukur dan dipantau melalui SLO Sloth
KNF-02	<i>Throughput</i> Tinggi	Sistem harus mampu menangani <i>workload</i> dengan <i>throughput</i> tinggi yang lazim pada beban kerja <i>streaming</i> dan <i>serving</i> bervolume tinggi, dengan karakteristik skalabilitas mendekati linear ketika sumber daya ditambah.	<i>Throughput ingest</i> dan <i>serving</i> tumbuh proporsional terhadap jumlah <i>partition</i> Kafka serta <i>replica</i> Feast melalui <i>auto-scaling</i> KEDA dan HPA
KNF-03	Skalabilitas Horizontal	Seluruh komponen sistem harus mendukung skala horizontal, sehingga kapasitas dapat ditingkatkan dengan menambah <i>node</i> dan menghasilkan peningkatan kinerja yang linear atau mendekati linear.	Seluruh <i>workload</i> terstandarisasi pada HPA (CPU/memori), VPA <i>recommender</i> , dan KEDA (Kafka <i>lag</i> , <i>custom metrics</i>)
KNF-04	Keandalan & Toleransi Kesalahan	Sistem harus andal dan tidak memiliki <i>single point of failure</i> , serta mampu melakukan pemulihan otomatis dari kegagalan sementara tanpa intervensi manual berlebihan.	SLO <i>platform</i> dijaga melalui Sloth (Kafka 99,9%, Postgres 99,95%, MinIO p99 <500ms pada 99%); <i>disaster recovery</i> berbasis Velero <i>snapshot</i> harian penuh dan <i>incremental</i> per <i>use case</i>
KNF-05	Konsistensi & Kebenaran	Sistem harus menjaga konsistensi data dan menjamin validitas temporal sehingga tidak terjadi kebocoran data yang merusak kebenaran hasil analisis maupun kinerja model.	<i>Zero tolerance</i> untuk kebocoran data temporal yang terdeteksi

ID	Kebutuhan Nonfungsional	Deskripsi	Target Metrik
KNF-06	<i>Observability</i>	Sistem harus memiliki <i>observability</i> menyeluruh, mencakup metrik, <i>structured log</i> , dan <i>distributed tracing</i> dengan dukungan OpenTelemetry, sehingga perilaku sistem dapat dianalisis end-to-end.	Seluruh komponen terinstrumentasi dengan metrik standar dan <i>distributed tracing</i> berbasis OpenTelemetry
KNF-07	Keamanan	Sistem harus menerapkan kontrol keamanan (otentikasi, otorisasi, enkripsi, dan <i>audit logging</i>) untuk mencegah akses tidak sah dan menjaga kerahasiaan data sensitif.	TLS 1.3 <i>in transit</i> , AES-256 <i>at rest</i>
KNF-08	Ekstensibilitas & Modularitas	Arsitektur harus modular sehingga komponen dapat diperluas atau diganti melalui antarmuka terdefinisi tanpa mengganggu komponen lain.	Komponen dapat diganti terpisah tanpa perubahan signifikan pada modul lain
KNF-09	Kemudahan Penggunaan	Antarmuka sistem harus intuitif dan didukung dokumentasi baik, sehingga pengguna baru dapat mengadopsi sistem dengan kurva belajar wajar.	Skor kepuasan pengguna > 4/5 pada survei internal
KNF-10	Efisiensi Sumber Daya	Sistem harus memanfaatkan sumber daya komputasi dan penyimpanan efisien untuk mengendalikan biaya sambil mempertahankan kinerja.	Kompresi kolumnar pada lapisan analitik dan <i>autoscaling</i> berbasis VPA/HPA-/KEDA agar utilisasi sumber daya proporsional terhadap beban; alokasi biaya dipantau oleh OpenCost
KNF-11	Portabilitas	Sistem harus dapat dijalankan di berbagai lingkungan <i>deployment</i> dengan perubahan terbatas pada konfigurasi dan tanpa modifikasi kode besar.	Dapat di- <i>deploy</i> di AWS, GCP, Azure, atau <i>on-premise</i> hanya dengan penyesuaian konfigurasi
KNF-12	Kemudahan Pemeliharaan	Sistem harus mudah dipelihara, dengan struktur kode jelas, cakupan pengujian memadai, dan proses <i>deployment</i> otomatis.	<i>Test coverage</i> > 80% pada komponen inti

III.3 Analisis Pemilihan Solusi

Bagian ini menyusun alternatif solusi yang dapat memenuhi kebutuhan fungsional dan nonfungsional, kemudian memilih solusi terbaik dengan menggunakan skor pemenuhan kebutuhan. Skor 0 berarti tidak terpenuhi, skor 1 berarti terpenuhi sebagai

an, dan skor 2 berarti terpenuhi penuh.

III.3.1 Alternatif Solusi

Empat alternatif solusi dipertimbangkan untuk membangun platform DataOps dan MLOps terintegrasi.

Alternatif A: Layanan Terkelola dari Penyedia *Cloud*. Pendekatan ini menggunakan rangkaian layanan terkelola pada satu penyedia, mencakup layanan ingestasi, *warehouse*, *feature store*, pelatihan, dan penyajian. Kelebihannya berupa kecepatan adopsi dan integrasi bawaan antar layanan, sebagaimana dibahas oleh Li dkk. (2023). Kekurangannya berupa keterikatan terhadap satu penyedia, biaya operasional pada beban tinggi, dan keterbatasan dalam menyesuaikan komponen pada level rendah.

Alternatif B: Tumpukan *Open Source* pada Kubernetes. Pendekatan ini merangkai komponen *open source* di atas Kubernetes sebagai bidang orkestrasi. Lakka (2025) dan Burgueno-Romero, Cánovas Izquierdo, dan García-García (2025) memperlihatkan rancangan serupa pada lingkungan *multi-cloud* dan *edge*. Kelebihannya berupa fleksibilitas, transparansi versi, portabilitas, dan kemampuan menegakkan pola GitOps. Kekurangannya berupa kebutuhan tata kelola yang lebih ketat dan kurva belajar yang lebih panjang dibandingkan layanan terkelola.

Alternatif C: Pengembangan Kustom dari Awal. Pendekatan ini membangun setiap lapisan secara mandiri tanpa banyak menggunakan komponen yang sudah jadi. Kelebihannya berupa kontrol penuh atas perilaku setiap komponen. Kekurangannya berupa biaya pengembangan dan pemeliharaan yang sangat besar, ketergantungan pada tim yang sangat kompeten, dan risiko menciptakan ulang masalah yang sudah diselesaikan oleh pustaka *open source* yang teruji.

Alternatif D: Hibrida antara Layanan Terkelola dan *Open Source*. Pendekatan ini menggunakan beberapa layanan terkelola, terutama pada lapisan dasar seperti penyimpanan dan jaringan, dipadukan dengan komponen *open source* pada lapisan DataOps dan MLOps. Kelebihannya berupa keseimbangan antara kecepatan adopsi dan fleksibilitas. Kekurangannya berupa kompleksitas integrasi antara dua jenis komponen dan kebutuhan kontrak antar lapisan yang harus dirancang dengan saksama.

III.3.2 Analisis Penentuan Solusi

Tabel III.3 menyajikan evaluasi pemenuhan kebutuhan fungsional dan Tabel III.4 menyajikan evaluasi pemenuhan kebutuhan nonfungsional. Skor total ditampilkan

pada baris terakhir Tabel III.4. Alternatif B memperoleh skor total tertinggi, yaitu 49 dari 52, diikuti oleh Alternatif A dan Alternatif D dengan skor 43, dan Alternatif C dengan skor 30.

Tabel III.3 Evaluasi Pemenuhan Kebutuhan Fungsional

Kebutuhan	Cloud	Open-Source	Kustom	Hibrida
KF-01: Manajemen Fitur Terpusat	2	2	1	2
KF-02: Penyajian Fitur Ganda	2	2	1	2
KF-03: Jaminan Validitas Temporal	1	2	2	1
KF-04: Dukungan <i>Vector Embeddings</i>	1	2	2	1
KF-05: Otomatisasi <i>Pipeline</i> Pelatihan	2	2	1	2
KF-06: Otomatisasi Penerapan Model	2	2	1	2
KF-07: Pemantauan <i>Drift</i> Otomatis	1	2	1	1
KF-08: <i>Governance</i> & Penemuan	2	2	1	2
KF-09: Pelacakan Eksperimen	2	2	1	2
KF-10: <i>Versioning</i> Registri Model	2	2	1	2
KF-11: Pemrosesan <i>Batch/Stream</i>	2	2	1	2
KF-12: Konfigurasi Deklaratif	2	2	1	2
KF-13: API dan SDK Terpadu	2	1	1	1
KF-14: Manajemen Sumber Daya	2	2	1	2
Total Skor KF	25	27	16	24

Tabel III.4 Evaluasi Pemenuhan Kebutuhan Nonfungsional

Kebutuhan	Cloud	Open-Source	Kustom	Hibrida
KNF-01: Kinerja Latensi Rendah	2	2	2	2
KNF-02: <i>Throughput</i> Tinggi	2	2	2	2
KNF-03: Skalabilitas Horizontal	2	2	1	2
KNF-04: Keandalan & Toleransi Kesalahan	2	2	1	2
KNF-05: Konsistensi & Kebenaran	1	2	2	1
KNF-06: <i>Observability</i>	2	2	1	2
KNF-07: Keamanan	2	2	1	2
KNF-08: Ekstensibilitas & Modularitas	0	2	2	1
KNF-09: Kemudahan Penggunaan	2	1	0	2
KNF-10: Efisiensi Sumber Daya	1	2	1	1
KNF-11: Portabilitas	0	2	1	1
KNF-12: Kemudahan Pemeliharaan	2	1	0	1
Total Skor KNF	18	22	14	19
TOTAL SKOR (KF+KNF)	43	49	30	43

Alternatif B unggul terutama pada kebutuhan yang menyangkut jaminan validitas temporal, dukungan *vector embeddings*, pemantauan *drift* otomatis, ekstensibili-

tas, dan portabilitas. Alternatif A unggul pada kemudahan penggunaan dan kemudahan pemeliharaan, tetapi tertinggal pada konsistensi temporal, dukungan *vector embeddings* pada *feature store*, dan portabilitas. Alternatif C unggul pada kontrol penuh tetapi kalah pada hampir semua kebutuhan operasional. Alternatif D mendekati Alternatif A tetapi tertinggal pada portabilitas dan ekstensibilitas. Berdasarkan skor pemenuhan kebutuhan dan analisis komparatif yang sejalan dengan Burgueno-Romero, Cánovas Izquierdo, dan García-García (2025), Amou Najabadi dkk. (2024), dan Lakka (2025), Alternatif B dipilih sebagai solusi yang akan dirancang pada Bab IV dan diimplementasikan pada Bab V.

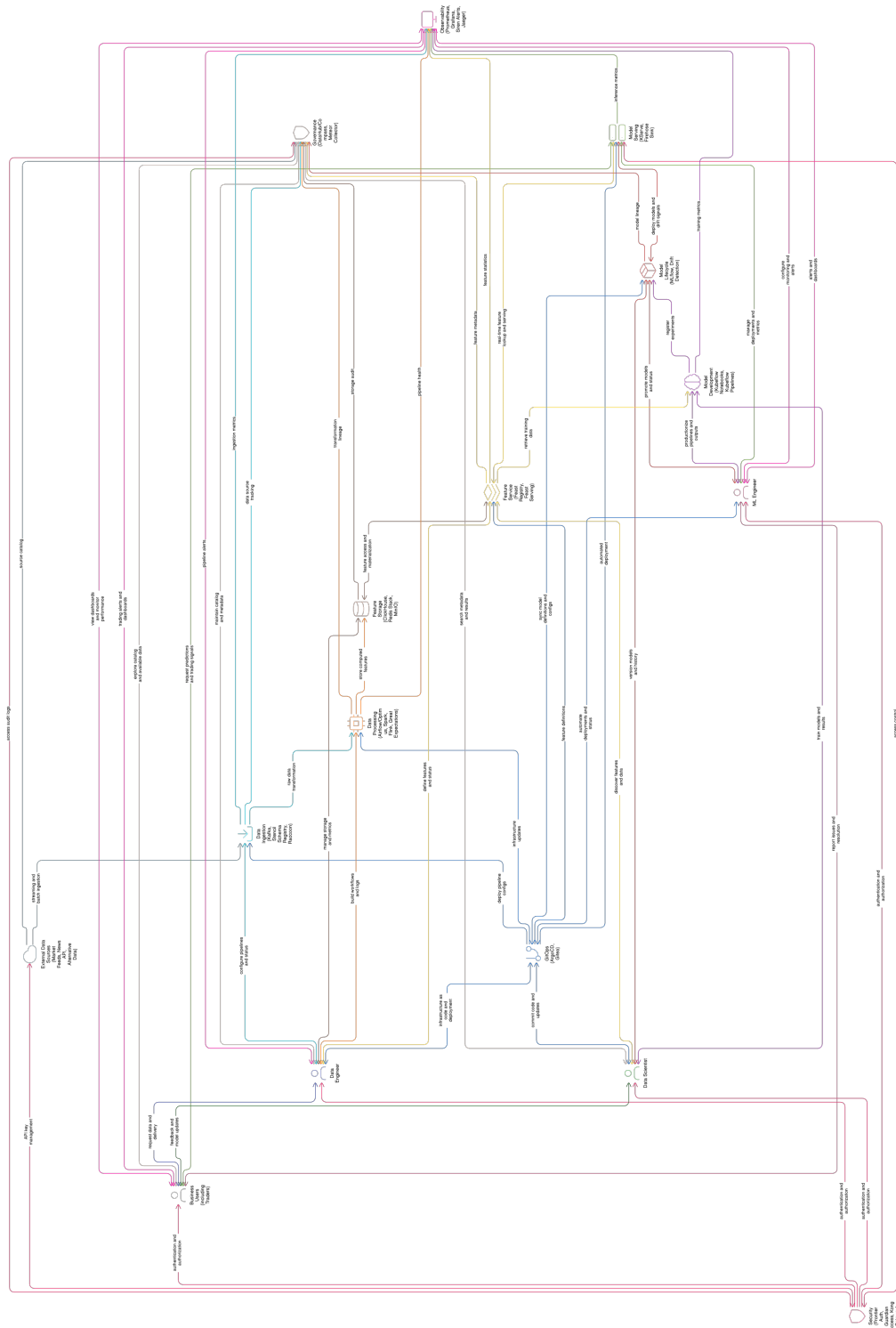
BAB IV

PERANCANGAN ARSITEKTUR PLATFORM DATAOPS DAN MLOPS TERINTEGRASI

Bab ini menyajikan perancangan arsitektur platform yang mengintegrasikan DataOps dan MLOps pada Kubernetes. Pembahasan dimulai dari gambaran umum yang merangkum perbedaan kondisi sebelum dan sesudah integrasi, kemudian dilanjutkan dengan perancangan empat sub-sistem yang masing-masing memenuhi satu tujuan dari Bab I: arsitektur platform terintegrasi sebagai pemenuhan T-1, sub-sistem tata kelola data sebagai pemenuhan T-2, sub-sistem deteksi *drift* dengan pelatihan ulang otomatis sebagai pemenuhan T-3, dan sub-sistem layanan fitur *dual-store* sebagai pemenuhan T-4. Bab ini menutup pembahasan dengan alur kerja *end-to-end* yang menghubungkan keempat sub-sistem dalam satu siklus reproduksibel.

IV.1 Gambaran Umum Platform

Platform yang dirancang menempatkan Kubernetes sebagai bidang orkestrasi tunggal dan menyatukan komponen DataOps dan MLOps di atasnya. Gambar IV.1 memperlihatkan arsitektur terintegrasi yang menggantikan kondisi fragmentasi pada Gambar III.5 dari Bab III. Komponen yang sebelumnya berdiri sendiri pada perangkat lunak yang berbeda kini terhubung melalui satu bidang kendali, satu sumber kebenaran berbasis Git, dan satu antarmuka observabilitas.



Gambar IV.1 Pandangan Umum Platform DataOps dan MLOps Terintegrasi

Perbedaan utama terhadap kondisi sebelum integrasi terletak pada tiga aspek. Pertama, bidang kendali yang sebelumnya tersebar pada bermacam *tool* digabungkan ke dalam *control plane* Kubernetes yang sama. Kedua, sumber kebenaran konfigurasi

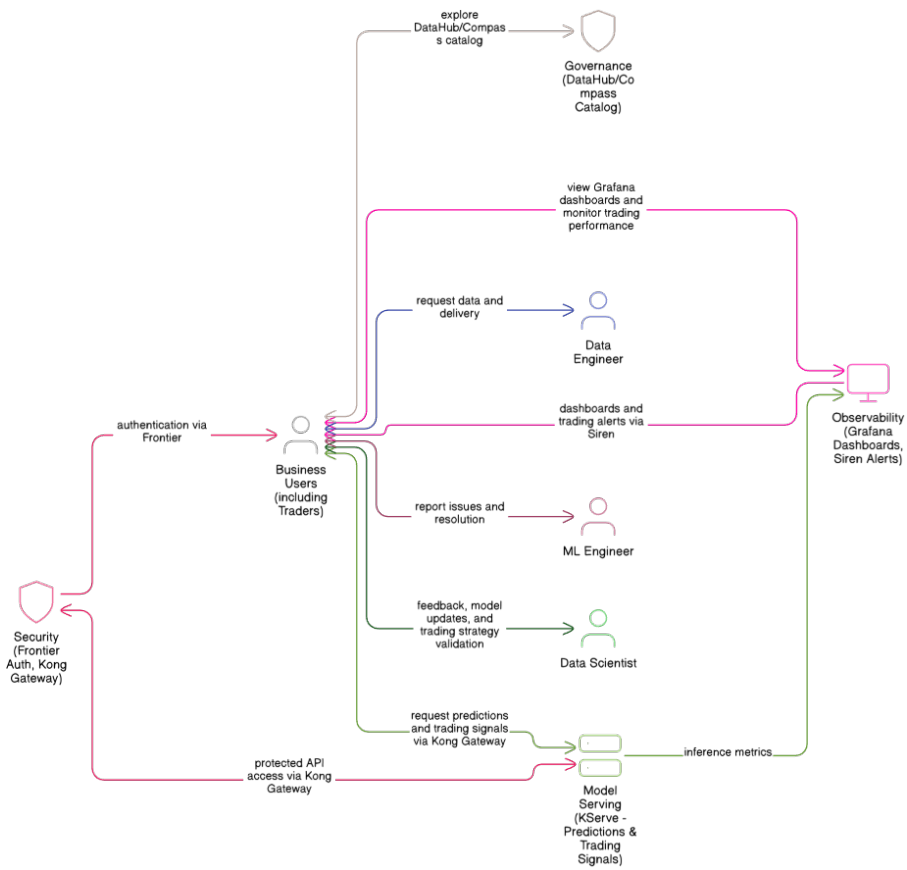
yang sebelumnya berada pada berkas konfigurasi terpisah disatukan pada repositori Git yang dijaga oleh Argo CD (Argo Project 2024). Ketiga, antarmuka observabilitas yang sebelumnya berbeda untuk metrik, log, dan *trace* disatukan di Grafana dengan Prometheus (Prometheus Authors 2024), Loki (Grafana Labs 2024b), dan Grafana Tempo (Grafana Labs 2024c) sebagai sumber tiga pilar observabilitas.

IV.2 Perancangan Arsitektur Terintegrasi

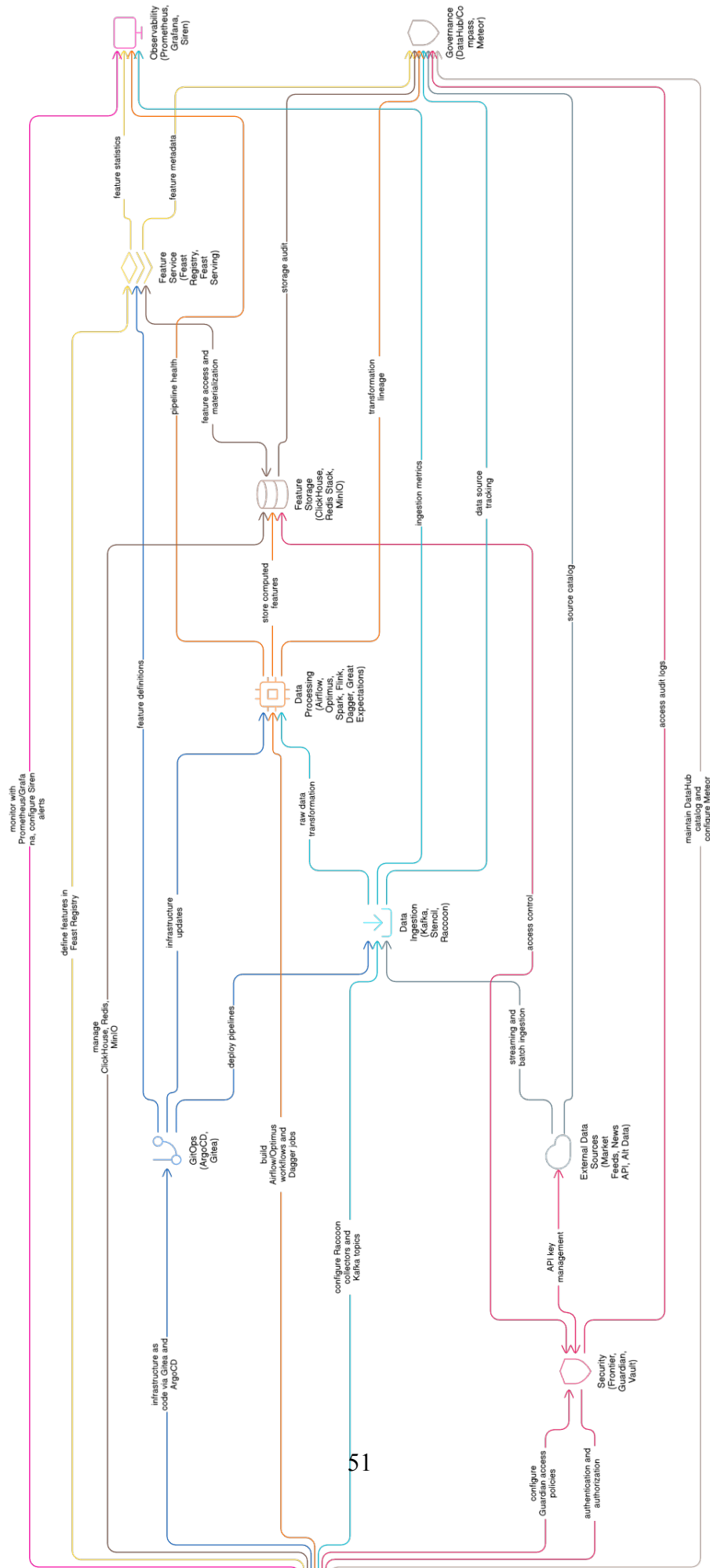
Bagian ini menjawab T-1 dengan merancang arsitektur platform DataOps dan MLOps terintegrasi pada Kubernetes. Arsitektur disusun dalam tujuh lapis yang saling terhubung melalui kontrak antarmuka eksplisit. Tujuh lapis yang dimaksud adalah lapisan infrastruktur, lapisan ingestasi data, lapisan pemrosesan, lapisan penyimpanan fitur, lapisan siklus hidup model, lapisan penyajian model, dan lapisan tata kelola serta observabilitas.

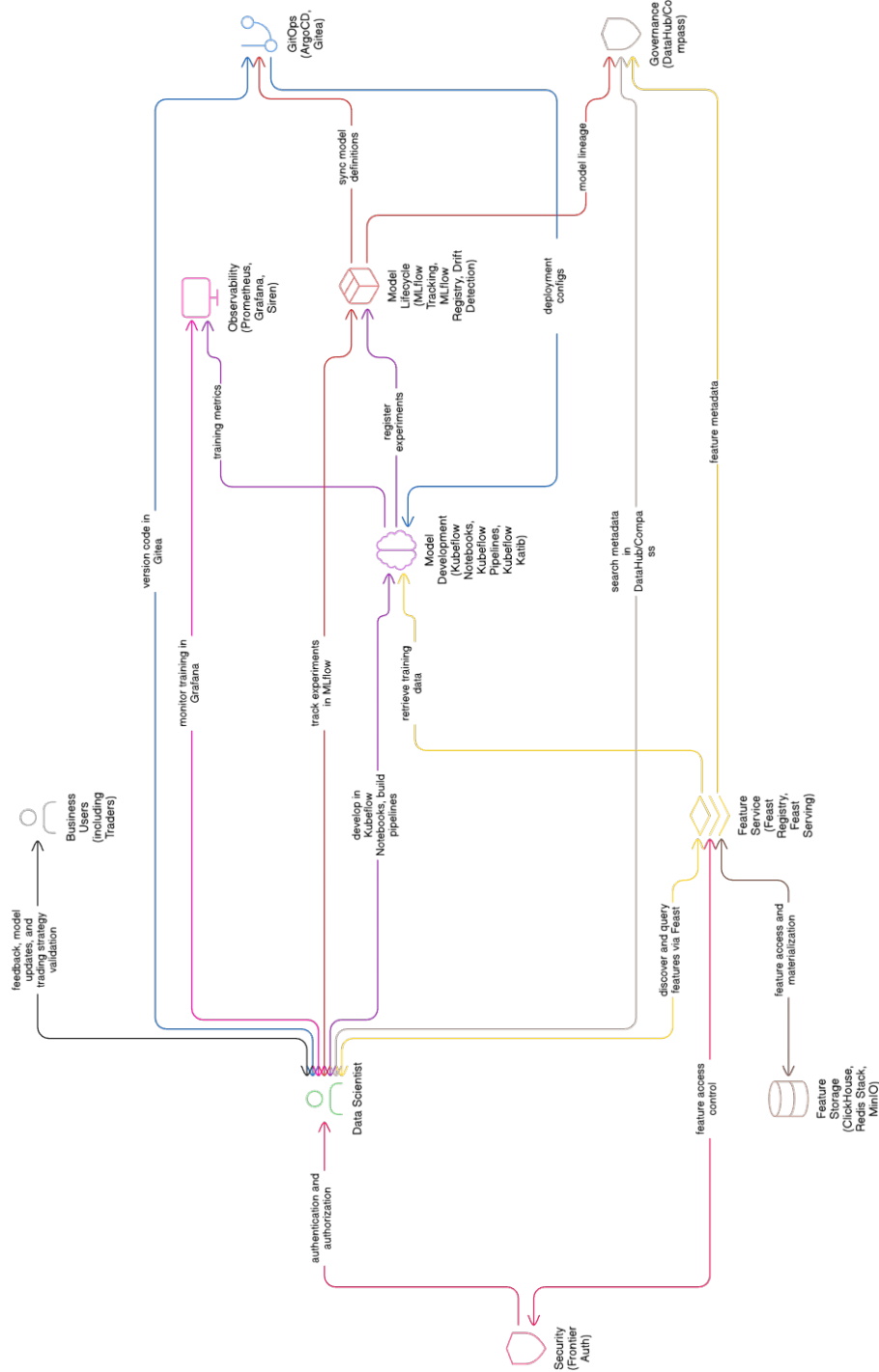
IV.2.1 Pandangan per Peran Pengguna

Empat peran utama mempunyai akses yang berbeda pada platform yang sama. Gambar IV.2 menyajikan pandangan untuk peran *business user*, Gambar IV.3 untuk peran *data engineer*, Gambar IV.4 untuk peran *data scientist*, dan Gambar IV.5 untuk peran *machine learning engineer*. Setiap peran berinteraksi dengan satu antarmuka yang sama, sehingga konteks dapat berpindah dari pelaporan ke katalog, dari katalog ke eksperimen, dan dari eksperimen ke penyebaran model tanpa berganti aplikasi.



Gambar IV.2 Pandangan Platform Terintegrasi untuk *Business User*





Gambar IV.4 Pandangan Platform Terintegrasi untuk *Data Scientist*

IV.2.2 Lapisan Infrastruktur dan Bidang Kendali

Lapisan infrastruktur menggunakan Kubernetes (Cloud Native Computing Foundation 2024a) sebagai bidang orkestrasi untuk seluruh komponen platform. Istio (Istio Authors 2024) menyediakan jaringan layanan dengan mTLS yang mengamankan komunikasi antar layanan, sedangkan APISIX (Apache APISIX Authors 2025) berperan sebagai *API gateway* pada perbatasan *mesh*. OpenBao (OpenBao Authors 2025) mengelola rahasia, dan *External Secrets Operator* menyebarkan rahasia dari OpenBao ke *namespace* target tanpa menyalin rahasia ke berkas *manifest*. Argo CD menjadi satu-satunya jalur perubahan, menarik konfigurasi dari Git dan menerapkan ke *cluster* melalui prinsip GitOps (Limoncelli 2022). Kebijakan keamanan dijalankan dalam dua lapis: Kyverno (Kyverno Authors 2024) pada lapis admisi Kubernetes, dan Falco (Falco Authors 2025) pada lapis *runtime*. Trivy Operator (Aqua Security 2025) memindai citra dan beban kerja secara terus menerus, sedangkan Velero (Velero Authors 2025) melakukan pencadangan sumber daya *cluster* dan *persistent volume* secara terjadwal.

IV.2.3 Lapisan Ingestasi dan Pemrosesan Data

Lapisan ingestasi menggunakan Apache Kafka (Apache Software Foundation 2024c) sebagai bus pesan utama. *Kafka Connect* digunakan untuk integrasi dengan sumber data eksternal, dan *Karapace* digunakan sebagai registri skema. Lapisan pemrosesan dibagi menjadi dua jalur. Jalur *batch* menggunakan Apache Spark (Apache Software Foundation 2024d) dan dbt untuk transformasi pada *data warehouse*. Jalur *stream* menggunakan Apache Flink (Apache Software Foundation 2024b) untuk pemrosesan jendela waktu, agregasi, dan deteksi pola. Penyimpanan jangka panjang pada *data lake* menggunakan MinIO sebagai *object store* dengan format Apache Iceberg sebagai *table format*, sedangkan Trino (Trino Software Foundation 2024) menyediakan kueri federasi pada beberapa sumber data sekaligus.

IV.2.4 Lapisan Siklus Hidup Model dan Penyajian

Lapisan siklus hidup model menggunakan MLflow (MLflow Contributors 2024) untuk *tracking* dan registri model. Kubeflow Pipelines (Kubeflow Contributors 2024) dipakai sebagai *orchestrator* pelatihan, sedangkan Katib digunakan untuk *hyperparameter tuning*. Lapisan penyajian menggunakan KServe (KServe Contributors 2024) sebagai *inference platform* dengan dukungan banyak *runtime*, otomatisasi penskalaan, dan integrasi langsung dengan model yang tersimpan pada registri MLflow. Argo Workflows mengeksekusi *pipeline* yang tidak terkait langsung dengan pelatihan, sedangkan Argo CronWorkflow menjadwalkan *job* berkala untuk pemeliharaan, validasi data, dan pemicu pelatihan ulang.

IV.2.5 Lapisan Tata Kelola dan Observabilitas

Lapisan tata kelola menggunakan DataHub (DataHub Project 2024) sebagai katalog metadata yang mengumpulkan informasi dari komponen lain melalui mekanisme *push* dan *pull*, dengan OpenLineage (OpenLineage Authors 2025) sebagai protokol pengirim peristiwa *lineage* dari komponen *pipeline*. Great Expectations (Great Expectations 2024) menjalankan pengujian kontrak data pada lapisan *batch* dan *stream*. Lapisan observabilitas menggunakan Prometheus untuk metrik, Loki untuk log, dan Grafana Tempo (Grafana Labs 2024c) untuk *trace*, dengan OpenTelemetry sebagai *collector* bersama. Grafana menyatukan ketiganya dan ditambahkan dasbor khusus untuk kualitas data, *drift* fitur, dan kinerja model. Sloth (Sloth Authors 2025) menu-runkan *service-level objective* ke aturan Prometheus, sedangkan Pyroscope menye-diakan profil performa berkelanjutan dan OpenCost menutup pemantauan biaya per beban kerja.

IV.3 Perancangan Sub-sistem Tata Kelola Data

Bagian ini menjawab T-2 dengan merancang sub-sistem tata kelola data yang konsis-ten lintas siklus data, fitur, dan model. Tata kelola yang dimaksud mencakup katalog metadata, *lineage*, pengujian kontrak data, dan kontrol akses berbasis identitas.

IV.3.1 Katalog Metadata dan *Lineage*

DataHub menjadi pusat katalog metadata. Sumber metadata datang dari beberapa komponen: Kafka untuk *topic* dan skema, ClickHouse (ClickHouse Inc 2024) un-tuk tabel pada *warehouse*, MinIO dan Apache Iceberg dengan katalog Lakekeeper (Lakekeeper Authors 2025) untuk *dataset* pada *data lake*, Feast (Feast Project 2024) untuk definisi fitur, MLflow untuk *experiment* dan model, serta Kubeflow Pipelines untuk *pipeline*. Sambungan *lineage* antar komponen dikirim melalui peristiwa Ope-nLineage (OpenLineage Authors 2025) sehingga DataHub dapat merangkai grafik ketertelusuran yang seragam dari ingestasi sampai prediksi. Identitas pengguna ka-talog dirambatkan dari *identity provider* pusat yang dipakai bersama oleh seluruh antarmuka platform.

IV.3.2 Kontrak Data dan Kualitas

Great Expectations memberi struktur untuk mendeklarasikan ekspektasi kualitas da-ta, termasuk tipe kolom, batas nilai, dan jumlah baris minimum. Ekspektasi dijalan-kan pada dua titik: setelah ingestasi pada lapisan *bronze* dan setelah transformasi pada lapisan *silver* dan *gold*. Hasil pengujian dipublikasikan ke DataHub sehingga kuali-tas data tertampil bersama metadata lain. Chien (February 2022) memberi panduan agar metrik kualitas dipakai sebagai dasar pengambilan keputusan, bukan sekadar

indikator pasif. Bernardo dkk. (2024) dan Stoyanovich, Howe, dan Jagadish (2020) memberi konteks yang melebihi aspek teknis, yakni pertanggungjawaban data sebagai bagian dari tata kelola platform.

IV.3.3 Identitas dan Kontrol Akses

Kontrol akses berbasis identitas menggunakan dex (Dex Authors 2025) sebagai *identity provider* OIDC dan oauth2-proxy (OAuth2 Proxy Authors 2025) sebagai *reverse proxy* yang menegakkan otentikasi pada antarmuka aplikasi. Identitas berlaku konsisten dari antarmuka pengguna ke *API* hingga ke akses data pada *warehouse* dan *lake*, dengan token OIDC yang sama dipakai oleh DataHub, MLflow, Grafana, Argo CD, dan Kubeflow Pipelines. Kebijakan otorisasi pada level kontainer dan beban kerja ditegakkan oleh Kyverno (Kyverno Authors 2024) pada lapis admisi Kubernetes, antara lain wajibnya *resource limits*, larangan *privileged container* di luar pengecualian eksplisit, dan kepatuhan label *Pod Security Standards*. Apa yang dibahas oleh Ahmad dkk. (June 2024) tentang strategi keamanan MLOps menjadi rujukan utama, karena banyak ancaman MLOps timbul dari ketidakhadiran kontrol akses yang konsisten di antara lapisan.

IV.4 Perancangan Sub-sistem Deteksi *Drift* dan *Continuous Training*

Bagian ini menjawab T-3 dengan merancang sub-sistem deteksi *drift* pada fitur dan label yang otomatis memicu pelatihan ulang pada saat melewati ambang batas. Sub-sistem ini menjawab kombinasi *drift* dan *temporal leakage* yang menjadi salah satu rumusan masalah.

IV.4.1 Pengukuran *Drift*

Dua indikator utama dipilih sebagai detektor. *Population Stability Index* (PSI) dipakai untuk fitur kategorikal dan kontinu dengan *binning*, sedangkan uji Kolmogorov-Smirnov dipakai untuk distribusi nilai kontinu (Reis dkk. 2016). Pengukuran dilakukan pada jendela waktu yang bergerak dan dibandingkan dengan jendela referensi. Tingkat *drift* dikategorikan menjadi *stable*, *moderate*, dan *severe* dengan ambang yang dapat dikonfigurasi per fitur. Bayram, Ahmed, dan Kassler (2022), Lu dkk. (2019), dan Hinder dkk. (2023) menjadi rujukan untuk penyusunan kebijakan keluar-ga detektor dan respons platform.

IV.4.2 Alur Pelatihan Ulang Otomatis

Alur pelatihan ulang dijalankan oleh Argo CronWorkflow yang berperan sebagai *control loop*. *Workflow* ini menjalankan *job* pengukuran *drift*, kemudian memutuskan apakah pemicu pelatihan diperlukan. Jika diperlukan, *workflow* memicu *pipeline* pelatihan pada Kubeflow Pipelines. *Pipeline* pelatihan menarik fitur dari Feast

dengan operasi `get_historical_features` yang menjamin *point-in-time correctness*, kemudian menulis model yang dihasilkan ke MLflow. Evidently (Evidently AI 2025) dijalankan sebagai pelapor pelengkap yang menghasilkan ringkasan distribusi fitur dan kinerja model dalam bentuk *report* yang dapat diaudit. Validasi kualitas model dijalankan sebelum penyebaran. Apabila lulus validasi, Argo Rollouts (Argo Rollouts Authors 2025) mengelola promosi versi baru pada KServe dengan strategi *canary* berbasis analisis metrik Prometheus, sehingga pengembalian otomatis dapat dilakukan ketika *signal* kinerja menurun. Céspedes Sisniega dkk. (2024) membeberikan contoh penerapan deteksi *drift* pada lingkungan *serverless*, dan pendekatan yang serupa diadaptasi ke lingkungan Kubernetes pada platform ini.

IV.4.3 Mekanisme Penanganan Kasus Khusus

Sub-sistem ini melayani tiga kasus khusus. Pertama, ketika *drift* terdeteksi pada fitur tetapi label belum tersedia, *workflow* hanya melakukan pelatihan ulang setelah label berjalan cukup lama. Kedua, ketika *drift* terjadi karena perubahan distribusi sementara, ambang dihitung dengan jendela bergulir agar tidak memicu pelatihan ulang yang tidak perlu. Ketiga, ketika model baru gagal lulus validasi, KServe mempertahankan versi lama dan *workflow* mencatat kegagalan ke MLflow.

IV.5 Perancangan Sub-sistem Layanan Fitur *Dual-Store*

Bagian ini menjawab T-4 dengan merancang sub-sistem layanan fitur yang menyatukan fitur tabular, fitur deret waktu, dan *embedding* berdimensi tinggi dalam satu kontrak data dari satu sumber kebenaran. Feast digunakan sebagai *framework* layanan fitur yang menyatukan kontrak antara *offline* dan *online*.

IV.5.1 Penyimpanan *Offline* dan *Online*

Offline store menggunakan ClickHouse untuk fitur tabular dan deret waktu yang dipakai pada fase pelatihan. ClickHouse dipilih karena kemampuan kompresi tinggi, latensi kueri rendah untuk *aggregation*, dan kompatibilitas dengan kontrak Feast. *Online store* menggunakan Valkey (Valkey Contributors 2025) untuk fitur yang dipakai pada fase *inference* dengan target latensi p99 di bawah sepuluh milidetik. Valkey dipilih karena kompatibilitas penuh dengan protokol RESP dan lisensi BSD pasca perubahan model lisensi Redis, sehingga klien Feast yang bersifat *drop-in* dapat dipakai tanpa modifikasi. *Vector search* HNSW tidak ditempelkan pada *online store* tabular, melainkan disediakan oleh Qdrant (Qdrant Team 2025) sebagai indeks vektor terpisah, sehingga beban kerja kunci-nilai dan beban kerja pencarian vektor terpisah secara operasional namun tetap berbagi kontrak fitur yang sama melalui Feast.

IV.5.2 Dukungan *Vector Embeddings* dan HNSW

Embedding dihasilkan oleh model `sentence-transformers/all-mpnet-base-v2` (Devlin dkk. 2019) berdimensi 768 untuk teks, dan disimpan sebagai fitur dengan dimensi yang konsisten antara fase pelatihan dan fase *inference*. Pencarian tetangga terdekat dijalankan oleh Qdrant (Qdrant Team 2025) dengan indeks HNSW (Malkov dan Yashunin 2020) berparameter `m=16` dan `ef_construct=200` yang memberi keseimbangan baik antara waktu kueri dan kualitas. Qdrant menyediakan kueri *gRPC* berlatensi rendah serta penyaringan *payload* pada saat pencarian, sehingga *embedding* dapat digabung dengan fitur tabular dari Valkey untuk membentuk *request inference* yang lengkap. Bruch, Gai, dan Ingber (2023) dan Campese, Lauriola, dan Moschitti (2024) memberi rujukan untuk fungsi peleburan hasil pencarian pada konteks *hybrid retrieval*.

IV.5.3 Jaminan *Point-in-Time Correctness*

Feast menjamin *point-in-time correctness* melalui operasi `get_historical_features`. Operasi ini melakukan *point-in-time join* antara entitas dan riwayat fitur dengan menggunakan `event_timestamp` dan `created_timestamp`, sehingga nilai fitur yang dipakai pada saat pelatihan identik dengan nilai yang akan tersedia pada saat *inference* di waktu yang sama. Vela dkk. (2022) dan Wang dan Ruf (2021) memperlihatkan dampak buruk *temporal leakage* pada validitas evaluasi model, dan Feast mencegah kebocoran tersebut secara struktural pada level kontrak data.

IV.5.4 Aliran Materialisasi Fitur

Materialisasi fitur dilakukan dengan dua jalur. Jalur *batch* menjalankan transformasi pada ClickHouse atau Spark, kemudian Feast memuat hasilnya ke *online store* secara periodik melalui operasi `materialize`. Jalur *stream* menggunakan Flink untuk menghitung fitur *real-time* yang ditulis langsung ke Valkey untuk fitur tabular dan ke Qdrant untuk *embedding* vektor, serta dipublikasikan ke ClickHouse pada saat yang sama, sehingga *offline* dan *online* tetap konsisten. Jain dan Das (2025) menyoroti pentingnya integrasi DataOps dengan MLOps untuk menjaga konsistensi seperti ini pada skala produksi.

IV.6 Alur Kerja *End-to-End*

Alur kerja menyatukan empat sub-sistem ke dalam satu siklus berkelanjutan. Pertama, data baru masuk melalui Kafka, divalidasi oleh Great Expectations, kemudian disimpan pada *warehouse* ClickHouse dan *data lake* MinIO dengan katalog Lake-keeper. Kedua, fitur dihitung pada Flink atau Spark, lalu diregistrasi pada Feast dan dimaterialisasi ke ClickHouse sebagai *offline store*, Valkey sebagai *online store*

tabular, dan Qdrant sebagai indeks vektor terpisah. Ketiga, Argo CronWorkflow mengukur *drift* dan, jika ambang dilewati, memicu *pipeline* pelatihan Kubeflow Pipelines. Pelatihan menggunakan `get_historical_features` untuk menjamin konsistensi temporal. Hasil pelatihan ditulis ke MLflow. Keempat, validasi model dijalankan, lalu Argo Rollouts mempromosikan versi KServe baru dengan strategi *canary* berbasis analisis metrik. Kelima, perubahan pada konfigurasi disinkronkan dari Git oleh Argo CD, sedangkan pengamatan kondisi sistem dipantau oleh Prometheus, Loki, dan Grafana Tempo melalui Grafana. Siklus ini berputar setiap kali data baru masuk atau *drift* terdeteksi, sehingga platform terus menerus menjaga akurasi model di lingkungan produksi. Rodrigues dkk. (2025) memberi rujukan tambahan untuk arsitektur pembelajaran *near real-time* pada lingkungan yang serupa.

BAB V

IMPLEMENTASI PLATFORM

Bab ini membahas implementasi platform DataOps dan MLOps terintegrasi sesuai rancangan pada Bab IV. Pembahasan dimulai dari lingkungan implementasi yang menjadi dasar penyebaran komponen, kemudian dilanjutkan dengan implementasi empat sub-sistem mengikuti urutan pemenuhan tujuan T-1 sampai T-4. Bab ini ditutup dengan verifikasi jalur eksekusi menggunakan satu *use case* sebagai instans pengujian. Penjabaran detail biner dan konfigurasi disimpan pada repositori platform, sehingga bab ini menyajikan pokok-pokok implementasi yang penting untuk membangun pemahaman atas bentuk akhir dari rancangan.

V.1 Lingkungan Implementasi

Lingkungan implementasi platform dibangun pada satu *cluster* Kubernetes (Cloud Native Computing Foundation 2024a) berbasis k3s yang berjalan pada satu *node*. Pilihan ini memenuhi kebutuhan eksekusi pada lingkungan terbatas tanpa mengubah pola penyebaran yang dapat dipakai pada *cluster* produksi dengan banyak *node*. Distribusi k3s membawa *control plane* ringan, *containerd* sebagai *runtime*, dan beberapa *add-on* bawaan yang dapat dinonaktifkan agar tidak berbenturan dengan komponen pilihan platform.

V.1.1 Konfigurasi Dasar *Cluster*

Cluster menjalankan komponen dasar berikut: Istio (Istio Authors 2024) sebagai *service mesh* dengan mTLS dalam mode *strict* pada lalu lintas internal, APISIX (Apache APISIX Authors 2025) sebagai *API gateway* pada perbatasan *mesh*, kontrol akses jaringan menggunakan *NetworkPolicy* pada level Kubernetes, dan Kyverno (Kyverno Authors 2024) sebagai *policy engine* pada lapis admisi. Falco (Falco Authors 2025) memantau perilaku *runtime* pada level *kernel*, sedangkan Trivy Operator (Aqua Security 2025) memindai citra dan beban kerja secara berkala. *Storage class* bawaan menggunakan *local-path provisioner* untuk *persistent volume* berbasis penyimpanan lokal, dengan Velero (Velero Authors 2025) melakukan pencadangan terjadwal ke MinIO. Chaos Mesh (Chaos Mesh Authors 2025) disiapkan untuk uji

ketahanan dengan eksperimen *fault injection* yang terbatas pada *namespace* tertentu. Konfigurasi sumber daya memberi prioritas pada satu replika *idle* per beban kerja, sementara *Horizontal Pod Autoscaler*, *Vertical Pod Autoscaler* dalam mode rekomendasi, dan KEDA *ScaledObject* disiapkan pada *template* agar penskalaan dapat aktif ketika beban nyata datang.

V.1.2 Manajemen Rahasia dan Identitas

OpenBao (OpenBao Authors 2025) bertindak sebagai penyimpan rahasia. *External Secrets Operator* bertugas menyebarkan rahasia dari OpenBao ke *namespace* target tanpa menyalin nilai rahasia ke dalam berkas *manifest*. Identitas pengguna dikelola oleh dex (Dex Authors 2025) sebagai *identity provider* OIDC, dengan oauth2-proxy (OAuth2 Proxy Authors 2025) sebagai *reverse proxy* yang menegakkan otentikasi pada antarmuka aplikasi seperti DataHub, MLflow, Grafana, Argo CD, dan Kube-flow Pipelines. Kebijakan otorisasi pada *cluster* dijalankan oleh Kyverno (Kyverno Authors 2024) pada lapis admisi, dengan aturan yang mewajibkan *resource limits*, melarang *privileged container* di luar pengecualian eksplisit, dan memaksa kepatuhan label *Pod Security Standards*.

V.1.3 Pola GitOps

Argo CD (Argo Project 2024) menjadi satu-satunya jalur perubahan pada *cluster*. Repositori Git pada Gitea (Gitea Contributors 2025) bertindak sebagai sumber kebenaran konfigurasi. Setiap perubahan dilakukan melalui *commit* pada repositori dan diterapkan ke *cluster* oleh Argo CD melalui rekonsiliasi. Pola ini menjamin reproduksibilitas dan menghasilkan jejak audit untuk setiap perubahan. *Pipeline* CI yang dijalankan oleh Tekton (Tekton Contributors 2024) membangun *image* dan menjalankan pengujian sebelum *commit* menjadi sumber penyebaran.

V.2 Implementasi Arsitektur Terintegrasi

Bagian ini mengimplementasikan rancangan pada Bagian IV.2. Komponen disusun ke dalam *namespace* per lapisan dan dihubungkan melalui kontrak *API* yang stabil.

V.2.1 Lapisan Ingestasi

Apache Kafka (Apache Software Foundation 2024c) diterapkan menggunakan operator Strimzi. *Kafka Connect* digunakan untuk integrasi dengan basis data eksternal melalui Debezium dan untuk *sink* ke Apache Iceberg pada MinIO (MinIO, Inc. 2024) dengan katalog Lakekeeper (Lakekeeper Authors 2025). *Karapace* berperan sebagai registri skema dengan dukungan Avro dan JSON Schema. Lalu lintas masuk dari konsumen di luar *mesh* dilewatkan melalui *API gateway* APISIX (Apache APISIX Authors 2025) dengan mTLS yang dijemputani pada perbatasan.

V.2.2 Lapisan Pemrosesan

Pemrosesan *stream* menggunakan operator Flink yang mengelola *FlinkDeployment*. Pemrosesan *batch* menggunakan operator Spark dan dijalankan sebagai *SparkApplication*. Transformasi pada *warehouse* menggunakan dbt yang dijalankan sebagai *Job* pada Tekton. Airflow (Apache Software Foundation 2024a) menjalankan *pipeline* data berorientasi *batch* yang tidak masuk ke dalam jalur pelatihan.

V.2.3 Lapisan Penyimpanan

Warehouse kolumnar menggunakan ClickHouse (ClickHouse Inc 2024) dengan operator ClickHouse pada satu *shard*. *Data lake* menggunakan MinIO sebagai *object store* dan Apache Iceberg sebagai *table format*, dengan Lakekeeper (Lakekeeper Authors 2025) sebagai katalog Iceberg REST dan Trino (Trino Software Foundation 2024) sebagai *query engine*. Penyimpanan *online* menggunakan Valkey (Valkey Contributors 2025) sebagai penyimpanan utama *online store* fitur tabular dengan protokol RESP, dan Qdrant (Qdrant Team 2025) sebagai indeks vektor terpisah untuk fitur *embedding* berdimensi tinggi. Penyimpanan relasional menggunakan PostgreSQL melalui operator CNPG, dan MySQL melalui operator yang sesuai untuk komponen yang memerlukan *engine* berbeda. Apache Superset (Apache Superset Authors 2025) disiapkan sebagai antarmuka BI untuk peran *business user* dengan koneksi ke ClickHouse, Trino, dan MySQL melalui SQLAlchemy.

V.2.4 Lapisan Siklus Hidup Model dan Penyajian

MLflow (MLflow Contributors 2024) dijalankan dengan basis data PostgreSQL dan *artifact store* pada MinIO. Kubeflow Pipelines (Kubeflow Contributors 2024) dijalankan pada *namespace* terpisah dengan dukungan *multi-tenant*, dilengkapi Kubeflow Notebooks untuk lingkungan eksperimen interaktif dan Kubeflow Trainer untuk pelatihan terdistribusi. KServe (KServe Contributors 2024) dipakai untuk *inference* dengan dukungan *runtime* MLflow, scikit-learn, dan ONNX, dan dengan Knative Serving (Knative Authors 2025) sebagai bidang *serverless* bawaan yang menyediakan penskalaan ke nol. Argo Workflows berperan sebagai *orchestrator job* di luar Kubeflow Pipelines, sedangkan Argo Rollouts (Argo Rollouts Authors 2025) mengelola promosi versi model dengan strategi *canary* dan *progressive analysis* berbasis metrik Prometheus.

V.2.5 Lapisan Observabilitas

Prometheus (Prometheus Authors 2024) mengumpulkan metrik dari semua komponen dengan *ServiceMonitor* dan *PodMonitor*. Loki (Grafana Labs 2024b) mengumpulkan log melalui Alloy. Grafana Tempo (Grafana Labs 2024c) menerima *tra-*

ce dari OpenTelemetry Collector yang tersebar pada *node*. Grafana (Grafana Labs 2024a) menggabungkan ketiganya pada satu antarmuka. Sloth (Sloth Authors 2025) menurunkan definisi *service-level objective* ke dalam aturan Prometheus sehingga *error budget* dapat dipantau bersama metrik lain. Pushgateway berperan sebagai titik kumpul metrik untuk *job* berdurasi pendek yang berjalan di luar siklus *scrape*. OpenCost dipakai untuk pemantauan biaya per beban kerja, sedangkan Pyroscope mengumpulkan profil *CPU* dan memori secara berkelanjutan.

V.3 Implementasi Sub-sistem Tata Kelola Data

Bagian ini mengimplementasikan rancangan pada Bagian IV.3.

V.3.1 Katalog Metadata dan Lineage

DataHub (DataHub Project 2024) dijalankan dengan komponen GMS, *frontend*, dan beberapa pekerjaan *ingest*, dengan OpenSearch sebagai *search backend* dan PostgreSQL sebagai *metadata backend*. *Job ingest* berjalan secara berkala sebagai *Cron-Job* untuk menarik metadata dari Kafka, ClickHouse, MinIO dan Iceberg, Feast, MLflow, dan Kubeflow Pipelines. Hasil *ingest* menyatukan *lineage* antar lapisan, mulai dari *topic* pada Kafka hingga model yang dilayani oleh KServe. OpenLineage (OpenLineage Authors 2025) dipakai sebagai protokol pengirim peristiwa *lineage* dari komponen *pipeline*, baik Airflow maupun Kubeflow Pipelines, agar grafik ketertelusuran di DataHub tersusun dengan kosa kata yang seragam.

V.3.2 Kontrak Data dan Kualitas

Great Expectations (Great Expectations 2024) mendefinisikan ekspektasi kualitas data pada lapisan *bronze* setelah ingestasi dan pada lapisan *silver* dan *gold* setelah transformasi. Setiap *expectation suite* disimpan pada repositori Git dan dijalankan oleh *CronJob* yang membaca hasil ke DataHub. Pengujian yang gagal akan menutup jalur *promotion* ke lapisan berikutnya dan memunculkan peringatan pada Grafana.

V.3.3 Kontrol Akses

Kontrol akses pada lapis jaringan menggunakan *NetworkPolicy* dengan kebijakan *default-deny* per *namespace* dan pengecualian yang dideklarasikan secara eksplisit. Kontrol akses pada lapis aplikasi menggunakan dex (Dex Authors 2025) sebagai *identity provider* OIDC dan oauth2-proxy (OAuth2 Proxy Authors 2025) sebagai *reverse proxy* yang menegakkan otentikasi pada antarmuka aplikasi. Kyverno (Kyverno Authors 2024) menerapkan kebijakan pada lapis admisi, antara lain wajibnya *resource limits*, larangan *privileged container* di luar pengecualian eksplisit, kepatuhan label, dan penegakan label *Pod Security Standards*. Falco (Falco Authors 2025) memantau peristiwa *runtime* pada level *kernel* untuk mendeteksi pelanggaran

kebijakan keamanan setelah *pod* berjalan.

V.4 Implementasi Sub-sistem Deteksi *Drift* dan *Continuous Training*

Bagian ini mengimplementasikan rancangan pada Bagian IV.4.

V.4.1 Detektor *Drift*

Detektor *drift* diimplementasikan sebagai *service* Python yang membaca fitur dari ClickHouse dengan jendela waktu yang dapat dikonfigurasi. Indikator PSI dihitung dengan *binning* adaptif, dan uji Kolmogorov-Smirnov dijalankan dengan ambang yang dapat disesuaikan per fitur (Reis dkk. 2016; Bayram, Ahmed, dan Kassler 2022; Lu dkk. 2019). Hasil pengukuran ditulis ke ClickHouse sebagai tabel `drift_multi_scale` dan dipublikasikan ke Pushgateway agar dapat diserap oleh Prometheus. Evidently (Evidently AI 2025) dijalankan sebagai pelapor pelengkap yang menghasilkan ringkasan distribusi fitur dan kinerja model dalam bentuk *report HTML* yang dapat diaudit dan dilampirkan pada katalog DataHub.

V.4.2 *Control Loop* Pelatihan Ulang

Control loop dijalankan oleh Argo CronWorkflow dengan jadwal yang dapat dikonfigurasi per model. *Workflow* menjalankan dua langkah utama: pengukuran *drift* dan pemicu *pipeline* pelatihan. Pemicu pelatihan mengirim permintaan ke Kube-flow Pipelines dengan parameter versi data dan versi model dasar. *Pipeline* pelatihan menarik fitur dari Feast dengan `get_historical_features`, menjalankan pelatihan pada *job* dengan sumber daya yang dialokasikan oleh Kueue (Kueue Authors 2024), menulis metrik ke MLflow, dan mendaftarkan model baru ke registri MLflow. Validasi model dijalankan sebelum penyebaran dengan membandingkan metrik kandidat terhadap model produksi pada *holdout set*. Jika lulus validasi, Argo Rollouts (Argo Rollouts Authors 2025) memperbarui *InferenceService* KServe dengan model baru memakai strategi *canary* dan analisis metrik Prometheus, sehingga pengembalian otomatis dapat dilakukan ketika kinerja menurun.

V.4.3 Penanganan Kasus Khusus

Tiga kasus khusus ditangani pada implementasi. Pertama, jendela tunggu label diatur per model agar pelatihan ulang tidak dipicu sebelum label tersedia cukup. Kedua, ambang *drift* dihitung dengan jendela bergulir untuk meredam fluktuasi singkat. Ketiga, ketika validasi gagal, KServe menahan versi lama dan *workflow* mencatat kejadian ke MLflow sebagai *run* yang gagal divalidasi.

V.5 Implementasi Sub-sistem Layanan Fitur *Dual-Store*

Bagian ini mengimplementasikan rancangan pada Bagian IV.5.

V.5.1 Konfigurasi Feast

Feast (Feast Project 2024) dikonfigurasi dengan `feature_store.yaml` yang menentukan *provider* Kubernetes, *offline store* ClickHouse, *online store* Valkey (Valkey Contributors 2025) melalui protokol RESP, dan registri *file-based* pada MinIO. Definisi fitur ditulis dalam Python pada modul `feature_definitions.py` dan didaftarkan ke registri melalui `feast apply`. Materialisasi fitur ke *online store* dijalankan oleh *job* berkala yang membaca dari ClickHouse dengan rentang waktu yang sesuai. Jain dan Das (2025) memperlihatkan pentingnya integrasi DataOps pada saat materialisasi seperti ini agar konsistensi antar lapisan tidak bergeser.

V.5.2 Dukungan Vector Embeddings

Vector embedding disimpan sebagai fitur dengan tipe `Array(Float32)` pada ClickHouse dan sebagai *point* berdimensi 768 pada *collection* Qdrant (Qdrant Team 2025). Pengindeksan HNSW diaktifkan pada Qdrant dengan parameter `m=16` dan `ef_construct=200` yang dapat dikonfigurasi melalui `HnswConfig`, dan kueri tetangga terdekat dilayani melalui *API gRPC* dengan filter *payload* pada saat pencarian. *Service vector-embedding* pada `platform/services/processing/vector/` membungkus model `sentence-transformers/all-mpnet-base-v2` sebagai penghasil *embedding* dan menyajikan *API* yang dipanggil oleh *stream processor* pada saat *event* masuk (Malkov dan Yashunin 2020; Devlin dkk. 2019).

V.5.3 Materialisasi dan Point-in-Time Correctness

`get_historical_features` dijalankan pada *pipeline* pelatihan dengan kolom `event_timestamp` pada *entity dataframe*. Feast melakukan *point-in-time join* pada ClickHouse menggunakan kueri dengan klausa `ASOF JOIN`, sehingga nilai fitur yang dipakai pada saat pelatihan identik dengan nilai yang tersedia pada saat itu (Vela dkk. 2022; Wang dan Ruf 2021). Materialisasi ke *online store* berjalan secara berkala dengan `feast materialize-incremental`, dan *stream materializer* pada Flink menulis fitur *real-time* ke Valkey untuk fitur tabular dan ke Qdrant untuk *embedding* secara bersamaan dengan penulisan ke ClickHouse.

V.6 Verifikasi Implementasi

Verifikasi jalur eksekusi dilakukan dengan satu *use case* pada domain pasar aset kripto sebagai instans pengujian. Pemilihan ini tidak mengubah sifat domain-agnostik dari platform, karena seluruh kode dan konfigurasi platform tetap berada pada repositori platform, sedangkan kode dan konfigurasi spesifik domain tinggal pada direktori `use-case-crypto/` yang terpisah dari platform. Direktori `use-case-crypto/services/` memuat tiga kelompok layanan: `websocket-collector/` sebagai produsen *event*

pasar, processing/ dengan subdirektori batch/, feature-store/, dan stream-processor/ sebagai pengolah dan penyaji fitur, serta dashboard/ dengan subdirektori backend/, frontend/, dan ml-bridge/ sebagai antarmuka pengguna dan jembatan *inference*. *Manifest* Kubernetes domain disimpan pada use-case-crypto/manifests/ dengan pola *base+overlays Kustomize*.

V.6.1 Lingkup Verifikasi

Verifikasi mencakup lima jalur. Pertama, jalur ingestasi diuji dengan *stream* dari sumber eksternal yang masuk ke Kafka melalui layanan websocket-collector pada *use case*. Kedua, jalur pemrosesan diuji dengan stream-processor berbasis Flink yang menulis fitur agregasi ke ClickHouse dan Valkey, serta *embedding* ke Qdrant. Ketiga, jalur pelatihan diuji dengan *pipeline* Kubeflow yang membaca fitur dari Feast dan menulis model ke MLflow. Keempat, jalur penyajian diuji dengan *InferenceService* KServe yang memuat model dari MLflow, dengan promosi versi yang dikelola oleh Argo Rollouts. Kelima, jalur pelatihan ulang diuji dengan menjalankan *drift-detector* dan memicu *retrain-on-drift workflow*. Uji ketahanan pendek menggunakan Chaos Mesh (Chaos Mesh Authors 2025) untuk menyuntikkan *fault* pada *pod* terpilih, dan pencadangan dijalankan terjadwal oleh Velero (Velero Authors 2025) ke MinIO sebagai langkah disiplin pemulihan.

V.6.2 Pengamatan Hasil Awal

Hasil pengamatan awal memperlihatkan bahwa kelima jalur berjalan sesuai rancangan. *End-to-end* jalur dari ingestasi sampai *inference* berhasil melayani permintaan, *point-in-time join* berhasil dijalankan pada pelatihan, dan *control loop* pelatihan ulang berhasil dipicu setelah *drift* buatan disuntikkan pada salah satu fitur. Penilaian kuantitatif terhadap latensi penyajian, *throughput* pemrosesan, dan tingkat ketepatan deteksi *drift* dilakukan pada tahap selanjutnya dan akan dilaporkan pada laporan akhir setelah seluruh tahap pengujian beban selesai.

BAB VI

EVALUASI

Bab ini akan diisi setelah seluruh komponen platform berjalan stabil pada *cluster* verifikasi dan pengamatan kuantitatif dapat dikumpulkan. Kerangka subbab dijamin konsisten dengan empat tujuan T-1 sampai T-4 sebagaimana didefinisikan pada Bab I agar setiap hasil evaluasi dapat dipetakan satu lawan satu kepada tujuan yang bersesuaian. Panduan gaya bahasa dan aturan penyajian tabel maupun gambar yang berlaku untuk seluruh laporan disimpan pada berkas `TEMPLATE_BAB.md` dan `WRITING_GUIDE.md`, sehingga panduan asli tetap dapat ditelusuri tanpa membebani isi bab evaluasi.

VI.1 Metode Evaluasi

Metode evaluasi akan dibagi menurut empat tujuan platform. Untuk T-1 evaluasi diarahkan pada reproduksibilitas penyebaran dari satu repositori Git melalui Argo CD, kelengkapan komponen yang *Healthy*, serta jejak kontrol versi pada *manifest* dan citra. Untuk T-2 evaluasi diarahkan pada kelengkapan *lineage* antara data, fitur, dan model pada katalog metadata, ketertelusuran pelanggaran kontrak kualitas, dan ketertegakan kebijakan akses. Untuk T-3 evaluasi diarahkan pada keberhasilan deteksi *drift* sintetis yang diinjeksikan secara terkendali, latensi siklus pelatihan ulang, dan kebebasan dari kebocoran temporal pada penyajian fitur *point-in-time*. Untuk T-4 evaluasi diarahkan pada kesetaraan nilai fitur antara jalur *offline* dan *online*, latensi pencarian fitur tabular maupun vektor, dan ketahanan layanan ketika beban meningkat.

VI.2 Hasil Evaluasi

Hasil pengukuran kuantitatif akan dilaporkan setelah platform menjalani periode pengamatan yang cukup pada *use case* verifikasi. Hasil tersebut akan mencakup metrik penyebaran (DORA *lead time*, *change failure rate*), metrik kualitas data (proporsi data yang lulus suite Great Expectations), metrik *drift* (PSI dan KS-test per fitur), metrik penyajian fitur (*latency* P95 dan P99 untuk fitur tabular dan vektor), serta metrik observabilitas (kelengkapan jejak OpenTelemetry dari *ingest* hingga predik-

si).

VI.3 Pembahasan Hasil Evaluasi

Pembahasan akan mengaitkan setiap hasil pengukuran dengan tujuan yang dijawab, mencatat keterbatasan, dan mengusulkan tindak lanjut. Pembahasan juga akan memeriksa apakah rantai kunci antara rumusan masalah, tujuan, perancangan, dan implementasi tetap konsisten setelah evaluasi.

BAB VII

PENUTUP

Bab ini akan diisi setelah hasil evaluasi pada Bab VI terkonsolidasi. Kerangka kesimpulan dijaga mengikuti aturan rantai kunci: setiap poin kesimpulan menjawab tepat satu tujuan T-1 sampai T-4 dengan urutan yang sama, tanpa menambahkan klaim baru di luar yang dibahas pada bab-bab sebelumnya.

VII.1 Kesimpulan

Kesimpulan akan dirangkum dalam empat poin yang bersesuaian dengan empat tujuan platform. Poin K1 akan merangkum bukti bahwa arsitektur DataOps dan MLOps terintegrasi pada Kubernetes dapat dipasang dari satu sumber Git dengan praktik GitOps. Poin K2 akan merangkum bukti bahwa katalog metadata, *lineage*, dan kontrol kualitas data berjalan otomatis sepanjang *pipeline* data, fitur, dan model. Poin K3 akan merangkum bukti bahwa deteksi *drift* terotomasi memicu pelatihan ulang sesuai kebijakan dan bahwa penyajian fitur memenuhi *point-in-time correctness*. Poin K4 akan merangkum bukti bahwa layanan fitur *dual-store* menyajikan fitur tabular, fitur agregat, dan fitur vektor dengan kontrak yang konsisten antara fase pelatihan dan fase *inference*.

VII.2 Saran

Saran akan diarahkan pada tindak lanjut untuk tujuan yang belum tertutup penuh oleh evaluasi awal serta pada arah pengembangan platform berikutnya. Beberapa arah yang telah teridentifikasi mencakup perluasan dukungan multi-*tenancy* pada lapis tata kelola, otomasi kebijakan kuota pada *Kueue* untuk beban pelatihan yang lebih besar, integrasi metrik biaya *OpenCost* dengan kebijakan *retraining*, dan adopsi profil keamanan yang lebih ketat melalui *Kyverno* setelah profil dasar matang.

DAFTAR PUSTAKA

- Adusumilli, Lakshmi Vara Prasad. 2025. “Serverless Kubernetes: The Evolution of Container Orchestration”. *European Journal of Computer Science and Information Technology* 13 (30): 20–36. <https://doi.org/10.37745/ejcsit.2013/vol13n302036>. <https://doi.org/10.37745/ejcsit.2013/vol13n302036>.
- Ahmad, Tazeem, Mohd Adnan, Saima Rafi, Muhammad Azeem Akbar, dan Ayesha Anwar. June 2024. “MLOps-Enabled Security Strategies for Next-Generation Operational Technologies”. Dalam *Proceedings of the 28th International Conference on Evaluation and Assessment in Software Engineering (EASE 2024)*, 662–670. <https://doi.org/10.1145/3661167.3661283>. <https://doi.org/10.1145/3661167.3661283>.
- Aiven Karapace Authors. 2025. *Karapace: Schema Registry and REST Proxy for Apache Kafka*. Diakses 25 Mei 2026. <https://www.karapace.io/>.
- Amazon Web Services. 2024. *What is an Event-Driven Architecture?* Diakses 15 Oktober 2025. <https://aws.amazon.com/event-driven-architecture/>.
- Amershi, Saleema, Andrew Begel, Christian Bird, Robert DeLine, Harald Gall, Ece Kamar, Nachiappan Nagappan, Besmira Nushi, dan Thomas Zimmermann. 2019. “Software Engineering for Machine Learning: A Case Study”. Dalam *2019 IEEE/ACM 41st International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, 291–300. IEEE. <https://doi.org/10.1109/ICSE-SEIP.2019.00042>. <https://doi.org/10.1109/ICSE-SEIP.2019.00042>.
- Amou Najafabadi, Faezeh, Justus Bogner, Ilias Gerostathopoulos, dan Patricia Lago. 2024. “An Analysis of MLOps Architectures: A Systematic Mapping Study”. Dalam *Software Architecture: 18th European Conference, ECSA 2024, Luxembourg City, Luxembourg, 3–6 September 2024, Proceedings*, 14889:69–85. Lecture Notes in Computer Science. Springer. https://doi.org/10.1007/978-3-031-70797-1_5. https://doi.org/10.1007/978-3-031-70797-1_5.

- Anaconda, Inc. July 2020. *2020 State of Data Science: Moving From Hype Toward Maturity*. Diakses 1 November 2025. <https://know.anaconda.com/rs/387-XNW-688/images/Anaconda-SODS-Report-2020-Final.pdf>.
- Apache APISIX Authors. 2025. *Apache APISIX: Cloud-Native API Gateway*. Diakses 10 November 2025. <https://apisix.apache.org/docs/>.
- Apache Software Foundation. 2024a. *Apache Airflow Documentation*. Diakses 5 November 2025. <https://airflow.apache.org/docs/>.
- . 2024b. *Apache Flink Documentation*. Diakses 30 September 2025. <https://flink.apache.org/documentation/>.
- . 2024c. *Apache Kafka Documentation*. Diakses 15 Oktober 2025. <https://kafka.apache.org/documentation/>.
- . 2024d. *Apache Spark Documentation*. Diakses 2 November 2025. <https://spark.apache.org/docs/latest/>.
- Apache Superset Authors. 2025. *Apache Superset Documentation*. Diakses 10 November 2025. <https://superset.apache.org/docs/intro>.
- Aqua Security. 2025. *Trivy Operator: Kubernetes Native Security Scanner*. Diakses 10 November 2025. <https://aquasecurity.github.io/trivy-operator/>.
- Argo Project. 2024. *Argo CD - Declarative GitOps CD for Kubernetes*. Diakses 8 November 2025. <https://argo-cd.readthedocs.io/>.
- Argo Rollouts Authors. 2025. *Argo Rollouts: Progressive Delivery for Kubernetes*. Diakses 10 November 2025. <https://argoproj.github.io/argo-rollouts/>.
- Bayram, Firas, Bestoun S. Ahmed, dan Andreas Kassler. 2022. “From Concept Drift to Model Degradation: An Overview on Performance-Aware Drift Detectors”. *Knowledge-Based Systems* 245:108632. <https://doi.org/10.1016/j.knosys.2022.108632>. <https://doi.org/10.1016/j.knosys.2022.108632>.
- Bernardo, B. M. V., H. S. Mamede, J. M. P. Barroso, dan V. M. P. D. dos Santos. 2024. “Data Governance and Quality Management: Innovation and Breakthroughs across Different Fields”. *Journal of Innovation and Knowledge* 9 (4): 100598. <https://doi.org/10.1016/j.jik.2024.100598>. <https://doi.org/10.1016/j.jik.2024.100598>.

- Bondi, Andre B. 2000. “Characteristics of Scalability and Their Impact on Performance”. Dalam *Proceedings of the 2nd International Workshop on Software and Performance*, 195–203. ACM. <https://doi.org/10.1145/350391.350432>.
- Bruch, Sebastian, Siyu Gai, dan Amir Ingber. 2023. “An Analysis of Fusion Functions for Hybrid Retrieval”. Dalam *ACM Transactions on Information Systems*. <https://doi.org/10.1145/3596512>.
- Burgueno-Romero, Anabel M., José L. Cánovas Izquierdo, dan José García-García. 2025. “Toward an Open Source MLOps Architecture”. *IEEE Software* 42 (1): 28–35. <https://doi.org/10.1109/MS.2024.3421675>. <https://doi.org/10.1109/MS.2024.3421675>.
- Campese, Stefano, Ivano Lauriola, dan Alessandro Moschitti. 2024. “Improving Document Retrieval Coherence for Semantically Equivalent Queries”. *arXiv preprint arXiv:2404.12338*.
- Carbone, Paris, Stephan Ewen, Kostas Tzoumas, Volker Markl, dan Fabian Hueske. 2015. “Apache Flink: Stream and Batch Processing in a Single Engine”. *Bulletin of the IEEE Computer Society Technical Committee on Data Engineering* 36 (4): 28–38. <https://asterios.katsifodimos.com/assets/publications/flink-deb.pdf>.
- Céspedes Sisniega, Jaime, Vicente Rodríguez, Germán Moltó, dan Álvaro López García. 2024. “Efficient and Scalable Covariate Drift Detection in Machine Learning Systems with Serverless Computing”. *Future Generation Computer Systems* 161:174–188. <https://doi.org/10.1016/j.future.2024.07.010>. <https://doi.org/10.1016/j.future.2024.07.010>.
- Chaos Mesh Authors. 2025. *Chaos Mesh: A Cloud-Native Chaos Engineering Platform*. Diakses 10 November 2025. <https://chaos-mesh.org/docs/>.
- Chien, Melody. February 2022. *How to Improve Your Data Quality*. Diakses 29 September 2025. Gartner. <https://www.gartner.com/smarterwithgartner/how-to-improve-your-data-quality>.
- ClickHouse Inc. 2024. *ClickHouse Documentation*. Diakses 5 November 2025. <https://clickhouse.com/docs/>.
- Cloud Native Computing Foundation. 2024a. *Kubernetes Documentation*. <https://kubernetes.io/docs/>.

- Cloud Native Computing Foundation. 2024b. *Observability Whitepaper*. Diakses 3 November 2025. <https://github.com/cncf/tag-observability/blob/main/whitepaper.md>.
- DataHub Project. 2024. *DataHub Documentation*. Diakses 5 November 2025. <https://datahubproject.io/docs/>.
- dbt Labs. 2025. *dbt: Data Build Tool Documentation*. Diakses 25 Mei 2026. <https://docs.getdbt.com/>.
- Devlin, Jacob, Ming-Wei Chang, Kenton Lee, dan Kristina Toutanova. 2019. “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding”. Dalam *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, 4171–4186. Association for Computational Linguistics. <https://doi.org/10.18653/v1/N19-1423>. <https://doi.org/10.18653/v1/N19-1423>.
- Dex Authors. 2025. *Dex: A Federated OpenID Connect Provider*. Diakses 10 November 2025. <https://dexidp.io/docs/>.
- Evidently AI. 2025. *Evidently: Open-Source ML Observability Platform*. Diakses 10 November 2025. <https://docs.evidentlyai.com/>.
- Falco Authors. 2025. *Falco: Cloud-Native Runtime Security*. Diakses 10 November 2025. <https://falco.org/docs/>.
- Feast Project. 2024. *Feast Documentation*. Diakses 5 November 2025. <https://docs.feast.dev/>.
- Gitea Contributors. 2025. *Gitea Documentation*. Diakses 10 November 2025. <https://docs.gitea.com/>.
- Google Cloud. 2024. *MLOps: Continuous Delivery and Automation Pipelines in Machine Learning*. Cloud Architecture Center. Diakses 2 November 2025. <https://cloud.google.com/architecture/mlops-continuous-delivery-and-automation-pipelines-in-machine-learning>.
- Grafana Labs. 2024a. *Grafana Documentation*. Diakses 5 November 2025. <https://grafana.com/docs/grafana/latest/>.
- . 2024b. *Grafana Loki Documentation*. Diakses 10 November 2025. <https://grafana.com/docs/loki/latest/>.

- . 2024c. *Grafana Tempo: High-Volume Distributed Tracing Backend*. Diakses 10 November 2025. <https://grafana.com/docs/tempo/latest/>.
- . 2025. *Grafana Pyroscope: Continuous Profiling*. Diakses 25 Mei 2026. <https://grafana.com/docs/pyroscope/latest/>.
- Great Expectations. 2024. *Great Expectations Documentation*. Diakses 10 November 2025. <https://docs.greatexpectations.io/docs/>.
- Hinder, Fabian, Valerie Vaquet, Johannes Brinkrolf, dan Barbara Hammer. 2023. “Model-Based Explanations of Concept Drift”. *Neurocomputing* 555:126640. <https://doi.org/10.1016/j.neucom.2023.126640>. <https://doi.org/10.1016/j.neucom.2023.126640>.
- Hsu, C. H., P. W. Liu, Y. C. Fan, dan C. Y. Lin. 2024. “End-to-End Automation of ML Model Lifecycle Management Using Machine Learning Operations Platforms”. Dalam *2024 International Conference on Consumer Electronics-Taiwan (ICCE-Taiwan)*, 1–6. IEEE. <https://doi.org/10.1109/ICCE-Taiwan62264.2024.10674445>. <https://doi.org/10.1109/ICCE-Taiwan62264.2024.10674445>.
- IBM. 2024. *What is Observability? Logs, Metrics, and Distributed Tracing*. Diakses 2 November 2025. <https://www.ibm.com/topics/observability>.
- Istio Authors. 2024. *Istio Service Mesh*. <https://istio.io/>.
- Jain, Souratn, dan Jyotipriya Das. 2025. “Integrating Data Engineering and MLOps for Scalable and Resilient Machine Learning Pipelines: Frameworks, Challenges, and Future Trends”. *World Journal of Advanced Engineering Technology and Sciences* 14 (1): 241–253. <https://doi.org/10.30574/wjaets.2025.14.1.0020>. <https://doi.org/10.30574/wjaets.2025.14.1.0020>.
- Jegou, Herve, Matthijs Douze, dan Cordelia Schmid. 2011. “Product Quantization for Nearest Neighbor Search”. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 33 (1): 117–128. <https://doi.org/10.1109/TPAMI.2010.57>. <https://doi.org/10.1109/TPAMI.2010.57>.
- Johnson, Jeff, Matthijs Douze, dan Hervé Jégou. 2021. “Billion-Scale Similarity Search with GPUs”. *IEEE Transactions on Big Data* 7 (3): 535–547. <https://doi.org/10.1109/TBDATA.2019.2921572>. <https://doi.org/10.1109/TBDATA.2019.2921572>.

- Jourand, Nicolas, Tom Bayer, Tobias Biegel, dan Joachim Metternich. 2023. “Handling Concept Drift in Deep Learning Applications for Process Monitoring”. *Procedia CIRP* 120:42–47. <https://doi.org/10.1016/j.procir.2023.08.007>. <https://doi.org/10.1016/j.procir.2023.08.007>.
- Kim, Gene, Jez Humble, Patrick Debois, dan John Willis. 2016. *The DevOps Handbook: How to Create World-Class Agility, Reliability, and Security in Technology Organizations*. 1st. IT Revolution Press. ISBN: 978-1942788003. <https://itrevolution.com/product/the-devops-handbook/>.
- Kleppmann, Martin. 2017. *Designing Data-Intensive Applications*. O’Reilly Media. ISBN: 978-1491903063. <https://www.oreilly.com/library/view/designing-data-intensive-applications/9781491903063/>.
- Knative Authors. 2025. *Knative Serving: Serverless on Kubernetes*. Diakses 10 November 2025. <https://knative.dev/docs/serving/>.
- Kreps, Jay. 2014. *Questioning the Lambda Architecture*. Diakses 8 November 2025. O’Reilly Radar. <https://www.oreilly.com/radar/questioning-the-lambda-architecture/>.
- Kreuzberger, Dominik, Niklas Kühl, dan Sebastian Hirschl. 2023. “Machine Learning Operations (MLOps): Overview, Definition, and Architecture”. *IEEE Access* 11:31866–31879. <https://doi.org/10.1109/ACCESS.2023.3262138>. <https://doi.org/10.1109/ACCESS.2023.3262138>.
- KServe Contributors. 2024. *KServe Documentation*. Diakses 12 Oktober 2025. <https://kserve.github.io/website/>.
- Kubeflow Contributors. 2024. *Kubeflow Documentation*. Diakses 29 September 2025. <https://www.kubeflow.org/docs/>.
- Kueue Authors. 2024. *Kueue: Job Queueing for Kubernetes*. Diakses 10 November 2025. <https://kueue.sigs.k8s.io/docs/>.
- Kyverno Authors. 2024. *Kyverno: Kubernetes Native Policy Management*. Diakses 10 November 2025. <https://kyverno.io/docs/>.
- Lakekeeper Authors. 2025. *Lakekeeper: Apache Iceberg REST Catalog*. Diakses 10 November 2025. <https://docs.lakekeeper.io/>.

- Lakka, Mounika. 2025. "Kubernetes for Enterprise: Mastering Cloud-Native Container Orchestration at Scale". *European Modern Studies Journal* 9 (3): 358–367. [https://doi.org/10.59573/emsj.9\(3\).2025.31](https://doi.org/10.59573/emsj.9(3).2025.31). [https://doi.org/10.59573/emsj.9\(3\).2025.31](https://doi.org/10.59573/emsj.9(3).2025.31).
- Lamer, Antoine, Chloé Saint-Dizier, Nicolas Paris, dan Emmanuel Chazard. 2024. "Data Lake, Data Warehouse, Datamart, and Feature Store: Their Contributions to the Complete Data Reuse Pipeline". *JMIR Medical Informatics* 12:e54590. <https://doi.org/10.2196/54590>. <https://doi.org/10.2196/54590>.
- Li, Anya, Bhala Ranganathan, Feng Pan, Mickey Zhang, Qianjun Xu, Runhan Li, Sethu Raman, Shail Paragbhai Shah, dan Vivienne Tang. 2023. "Managed Geo-Distributed Feature Store: Architecture and System Design". *arXiv preprint arXiv:2305.20077*, <https://doi.org/10.48550/arXiv.2305.20077>. <https://arxiv.org/abs/2305.20077>.
- Liang, Penghao, Wei Chen, Yifan Zhang, dan Jun Liu. 2024. "Automating the Training and Deployment of Models in MLOps by Integrating Systems with Machine Learning". *arXiv preprint arXiv:2405.09819*, <https://doi.org/10.48550/arXiv.2405.09819>. <https://arxiv.org/abs/2405.09819>.
- Limoncelli, Thomas. 2022. "GitOps: A Path to More Self-Service IT". *Communications of the ACM* 65 (8): 43–49. <https://doi.org/10.1145/3233241>. <https://dl.acm.org/doi/10.1145/3233241>.
- Lu, Jie, Anjin Liu, Fan Dong, Feng Gu, Joao Gama, dan Guangquan Zhang. 2019. "Learning Under Concept Drift: A Review". Versi cetak: *arXiv:2004.05785* (2020), *IEEE Transactions on Knowledge and Data Engineering* 31 (12): 2346–2363. <https://arxiv.org/abs/2004.05785>.
- Malkov, Yury A., dan D. A. Yashunin. 2020. "Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs". *IEEE Transactions on Pattern Analysis and Machine Intelligence* 42 (4): 824–836. <https://doi.org/10.1109/TPAMI.2018.2889473>. <https://doi.org/10.1109/TPAMI.2018.2889473>.
- Marz, Nathan, dan James Warren. 2015. *Big Data: Principles and Best Practices of Scalable Realtime Data Systems*. Manning Publications. ISBN: 978-1617290343. <https://www.manning.com/books/big-data>.

- McKnight, William. January 2020. *Delivering on the Vision of MLOps: A Maturity-Based Approach*. Diakses 2 November 2025. GigaOm. <https://gigaom.com/report/delivering-on-the-vision-of-mlops/>.
- Microsoft Azure. 2023. *Machine Learning Operations Maturity Model*. Azure Architecture Center. Diakses 2 November 2025. <https://learn.microsoft.com/en-us/azure/architecture/ai-ml/guide/mlops-maturity-model>.
- MinIO, Inc. 2024. *MinIO Documentation*. Diakses 5 November 2025. <https://min.io/docs/>.
- MLflow Contributors. 2024. *MLflow Documentation*. Diakses 1 November 2025. <https://mlflow.org/docs/latest/index.html>.
- Munappy, Aiswarya Raj, Jan Bosch, Helena Holmström Olsson, Anders Arpteg, dan Björn Brinne. 2022. "Data Management for Production Quality Deep Learning Models: Challenges and Solutions". *Journal of Systems and Software* 191:111359. <https://doi.org/10.1016/j.jss.2022.111359>. <https://doi.org/10.1016/j.jss.2022.111359>.
- OAuth2 Proxy Authors. 2025. *OAuth2 Proxy Documentation*. Diakses 10 November 2025. <https://oauth2-proxy.github.io/oauth2-proxy/>.
- Open Policy Agent Authors. 2024. *Open Policy Agent*. <https://www.openpolicyagent.org/>.
- OpenAI. December 2022. *New and Improved Embedding Model*. OpenAI Blog. Diakses 2 November 2025. <https://openai.com/index/new-and-improved-embedding-model/>.
- OpenBao Authors. 2025. *OpenBao: Open Source Secrets Management*. Diakses 10 November 2025. <https://openbao.org/docs/>.
- OpenCost Authors. 2025. *OpenCost: Open Source Cost Monitoring for Kubernetes*. Diakses 25 Mei 2026. <https://www.opencost.io/docs/>.
- OpenLineage Authors. 2025. *OpenLineage: An Open Standard for Lineage Metadata Collection*. Diakses 10 November 2025. <https://openlineage.io/docs/>.
- Paleyes, Andrei, Raoul-Gabriel Urma, dan Neil D. Lawrence. 2022. "Challenges in Deploying Machine Learning: A Survey of Case Studies". *ACM Computing Surveys* 55 (6): 1–29. <https://doi.org/10.1145/3533378>. <https://doi.org/10.1145/3533378>.

- Peppers, Ken, Tuure Tuunanen, Marcus A. Rothenberger, dan Samir Chatterjee. 2007. "A Design Science Research Methodology for Information Systems Research". *Journal of Management Information Systems* 24 (3): 45–77. <https://doi.org/10.2753/MIS0742-1222240302>. <https://doi.org/10.2753/MIS0742-1222240302>.
- Pimentel, João Felipe, Leonardo Murta, Vanessa Braganholo, dan Juliana Freire. 2019. "A Large-Scale Study About Quality and Reproducibility of Jupyter Notebooks". Dalam *2019 IEEE/ACM 16th International Conference on Mining Software Repositories (MSR)*, 507–517. IEEE. <https://doi.org/10.1109/MSR.2019.00077>. <https://doi.org/10.1109/MSR.2019.00077>.
- Polyzotis, Neoklis, Sudip Roy, Steven Euijong Whang, dan Martin Zinkevich. 2018. "Data Lifecycle Challenges in Production Machine Learning: A Survey". *ACM SIGMOD Record* 47 (2): 17–28. <https://doi.org/10.1145/3299887.3299891>. <https://doi.org/10.1145/3299887.3299891>.
- Prometheus Authors. 2024. *Prometheus Documentation*. Diakses 5 November 2025. <https://prometheus.io/docs/>.
- . 2025. *Prometheus Pushgateway*. Diakses 25 Mei 2026. <https://prometheus.io/docs/practices/pushing/>.
- Qdrant Team. 2025. *Qdrant: High-Performance Vector Search Engine*. Diakses 25 Mei 2026. <https://qdrant.tech/documentation/>.
- Reis, Denis Moreira dos, Peter Flach, Stan Matwin, dan Gustavo Batista. 2016. "Fast Unsupervised Online Drift Detection Using Incremental Kolmogorov-Smirnov Test". Dalam *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 1545–1554. ACM. <https://doi.org/10.1145/2939672.2939836>. <https://doi.org/10.1145/2939672.2939836>.
- Rella, Bhanu Prakash Reddy. 2022. "MLOps and DataOps Integration for Scalable Machine Learning Deployment". *International Journal for Multidisciplinary Research* 4 (1). <https://www.ijfmr.com/papers/2022/1/39278.pdf>.
- Rodrigues, Miguel G., Eduardo K. Viegas, Altair O. Santin, dan Fabrício Enembreck. 2025. "A MLOps Architecture for Near Real-Time Distributed Stream Learning Operation Deployment". *Journal of Network and Computer Applications* 238:104169. <https://doi.org/10.1016/j.jnca.2025.104169>. <https://doi.org/10.1016/j.jnca.2025.104169>.

- Sculley, D., Gary Holt, Daniel Golovin, Eugene Davydov, Todd Phillips, Dietmar Ebner, Vinay Chaudhary, Michael Young, Jean-Francois Crespo, dan Dan Dennison. 2015. “Hidden Technical Debt in Machine Learning Systems”. Dalam *Advances in Neural Information Processing Systems 28 (NIPS 2015)*, disunting oleh C. Cortes, N. D. Lawrence, D. D. Lee, M. Sugiyama, dan R. Garnett, 2503–2511. Red Hook, NY: Curran Associates, Inc. <https://proceedings.neurips.cc/paper/2015/file/86df7dcfd896fcaf2674f757a2463eba-Paper.pdf>.
- Shankar, Shreya, Rolando Garcia, Joseph M. Hellerstein, dan Aditya G. Parameswaran. 2022. *Operationalizing Machine Learning: An Interview Study*. ArXiv preprint arXiv:2209.09125. <https://doi.org/10.48550/arXiv.2209.09125>. <https://arxiv.org/abs/2209.09125>.
- Sloth Authors. 2025. *Sloth: Easy and Reliable SLOs for Prometheus*. Diakses 10 November 2025. <https://sloth.dev/>.
- Stopford, Ben. 2018. *Designing Event-Driven Systems: Concepts and Patterns for Streaming Services with Apache Kafka*. O’Reilly Media. ISBN: 978-1-4920-3819-5.
- Stoyanovich, Julia, Bill Howe, dan H. V. Jagadish. 2020. “Responsible Data Management”. *Proceedings of the VLDB Endowment* 13 (12): 3474–3488. <https://doi.org/10.14778/3415478.3415570>. <https://doi.org/10.14778/3415478.3415570>.
- Tekton Contributors. 2024. *Tekton Pipelines Documentation*. Diakses 10 November 2025. <https://tekton.dev/docs/>.
- Trino Software Foundation. 2024. *Trino: Distributed SQL Query Engine*. <https://trino.io/>.
- Valkey Contributors. 2025. *Valkey: An Open Source In-Memory Data Store*. Diakses 25 Mei 2026. <https://valkey.io/docs/>.
- Vela, Daniel, Andrew Sharp, Richard Zhang, Trang Nguyen, An Hoang, dan Oleg S. Pianykh. 2022. “Temporal Quality Degradation in AI Models”. *Scientific Reports* 12:11654. <https://doi.org/10.1038/s41598-022-15245-z>. <https://doi.org/10.1038/s41598-022-15245-z>.
- Velero Authors. 2025. *Velero: Backup and Migrate Kubernetes Resources and Persistent Volumes*. Diakses 10 November 2025. <https://velero.io/docs/>.

- Wang, Weiquan, dan Johannes Ruf. 2021. "Information Leakage in Backtesting". *SSRN Electronic Journal*, <https://doi.org/10.2139/ssrn.3836631>. <https://doi.org/10.2139/ssrn.3836631>.
- Zaharia, Matei, Reynold S. Xin, Patrick Wendell, Tathagata Das, Michael Armbrust, Ankur Dave, Xiangrui Meng, dkk. 2016. "Apache Spark: A Unified Engine for Big Data Processing". *Communications of the ACM* 59 (11): 56–65. <https://doi.org/10.1145/2934664>. <https://doi.org/10.1145/2934664>.

LAMPIRAN A

DAFTAR ARTEFAK PLATFORM

Lampiran ini merangkum lokasi dan tata letak artefak yang dirujuk oleh Bab IV dan Bab V. Kode program lengkap, *manifest* Kubernetes, *chart* Helm, dan berkas konfigurasi disimpan pada repositori Gitea internal yang menjadi sumber kebenaran bagi Argo CD. Berkas terkait *use case* pasar aset kripto sebagai instans uji disimpan pada repositori terpisah agar tata kelola domain tidak bercampur dengan *control plane* platform.

A.1 Tata Letak Repositori Platform

Repositori platform disusun mengikuti pemisahan komponen menjadi tujuh lapisan sebagaimana digambarkan pada Subbab IV.1. Direktori `platform/components/` memuat *manifest* setiap komponen dasar yang dipasang melalui Argo CD; direktori `platform/services/` memuat layanan kustom yang ditulis untuk kebutuhan tata kelola, deteksi *drift*, dan layanan fitur; direktori `platform/scripts/` memuat *script* bantu untuk bootstrap, nuke, dan penskalaan beban.

A.2 Tata Letak Repositori Use Case

Repositori *use case* pasar aset kripto memuat berkas *overlay* Kustomize yang mengikat *base manifest* platform dengan konfigurasi domain. Berkas `config/project.yaml` menyimpan parameter spesifik domain yang dirujuk oleh Tekton, Argo Workflows, dan Feast. Direktori `services/` memuat kode produsen data, transformator fitur, dan *trainer* yang dipanggil oleh *pipeline* pelatihan.

A.3 Contoh Berkas Konfigurasi Feast

Berkas `feature_store.yaml` mendeklarasikan *registry file-based* pada MinIO, *offline store* pada ClickHouse, *online store* fitur tabular pada Valkey melalui protokol RESP, dan indeks vektor pada Qdrant secara terpisah. Berkas ini diolah oleh layanan fitur pada saat *materialization* batch maupun *streaming*; rincian sintaks dijelaskan pada Subbab V.5.

LAMPIRAN B

HASIL VERIFIKASI USE CASE

Lampiran ini akan diisi dengan tangkapan layar dan keluaran perintah yang membuktikan jalur ujung ke ujung pada *use case* pasar aset kripto setelah evaluasi pada Bab VI terkonsolidasi. Bukti yang direncanakan meliputi:

1. Status sinkronisasi Argo CD untuk seluruh *Application* platform dan *use case* pada kondisi *Synced/Healthy*.
2. Hasil eksekusi *pipeline* pelatihan dari Kubeflow Pipelines beserta *run id* pada MLflow.
3. Hasil eksekusi *CronWorkflow retrain-on-drift* ketika ambang batas dilewati.
4. Hasil panggilan `get_online_features` terhadap Feast untuk fitur tabular dan fitur vektor.
5. Hasil panggilan prediksi terhadap *InferenceService* KServe pada *model-serving* dengan respons HTTP yang valid.
6. Cuplikan dasbor Grafana yang menunjukkan kelengkapan jejak *OpenTelemetry* dari ingestasi hingga prediksi.

